“Question Bank Management System”

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

S. PREM SAI **:2520030561**

D. MOHIT VENKAT SAI **:2520030106**

Team Number: 23

Section: 8

Under the Guidance of

Dr. V. Sireesha
Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled **"Question Bank Management System"** is a record of Bonafide work of **S. Prem Sai – 2520030561, D. Mohit Venkat Sai – 2520030106** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

S. Prem Sai – 2520030561

D. Mohit Venkat Sai – 2520030106

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled **"Question Bank Management System"** is a bonafide work done and submitted by **S. PREM SAI – 2520030561, D. MOHIT VENKAT SAI – 2520030106** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27.**

Signature of the Guide
(Dr. V. Sireesha)

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, **Dr. V. Sireesha**, Professor, Department of Computer Science and Engineering, for her guidance, encouragement and constant support throughout this project. Her suggestions during the reviews helped us shape the scope of the Question Bank Management System and improve both the implementation and this report.

We are thankful to **Dr. P. Venkateshwara Rao**, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the facilities and an encouraging academic environment that made this work possible.

We also thank the management of Koneru Lakshmaiah Education Foundation, the faculty members and the course coordinators of Data Structures and Algorithms-3 (25CS2103E), and our classmates and families for their help and motivation during the course of this project.

S. Prem Sai – 2520030561

D. Mohit Venkat Sai – 2520030106

Abstract

Universities keep growing banks of examination questions across many subjects, units and years. Faculty who set a paper must find suitable questions quickly, avoid repeating questions that appeared before, and still find the right question when they misspell a keyword. Plain substring search does not handle these needs well. The **Question Bank Management System (QBMS)** is a Data Structures and Algorithms project that stores, searches and checks examination questions using classical string-matching algorithms.

The system is built as two clients that share the same algorithms. A **Java console application** provides add, view, edit and delete operations and an eleven-option menu for search and analysis. A **browser-based web application** (HTML, CSS and JavaScript) adds role-based login for students and administrators, question documents, JSON import and export, filters by subject, unit, difficulty and year, a step-by-step visualiser for each algorithm, and an exam-paper generator. Exact keyword search uses **Knuth-Morris-Pratt (KMP)**, **Rabin-Karp** and, in the web client, **Boyer-Moore-Horspool**. Several keywords are searched in one pass with an **Aho-Corasick** automaton. Typo-tolerant search, pairwise similarity and duplicate detection use **Levenshtein edit distance** computed by dynamic programming.

We tested the system on a corpus of 51 questions in four JSON files (DSA, Operating Systems, DBMS and Computer Networks), which gives 74 questions after merging with the built-in sets. In 100,000 or more random test cases, every search algorithm returned exactly the same matches as a brute-force reference. On a 102,000-question scaled corpus, Aho-Corasick searched three keywords about 4.4 times faster than running KMP once per keyword. Duplicate detection at a 60% threshold flagged seven question pairs, six of which were genuine repeats across question sets. The report also records the system's current limitations and the improvements planned for the final review.

Table of Contents

			Page No.
	Acknowledgement		
	Abstract		
	Table of Contents		
	List of Figures		
	List of Tables		
1	Introduction	1.1 Overview 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives	1–3
2	System Requirements	2.1 Functional Requirements 2.2 Non-Functional Requirements 2.3 Hardware and Software Requirements	4–5
3	System Design	3.1 High-level Architecture Diagram 3.2 Description of Each Layer 3.3 Class Design 3.4 Control Flow	6–9
4	Implementation	4.1 Module-wise Implementation 4.2 Front-end Logic (UI Rendering and State Management) 4.3 API Integration 4.4 Import, Export and Paper Generator 4.5 Complexity Summary 4.6 How to Run	10–16
5	Results	5.1 Test Environment and Dataset 5.2 Java Console Application 5.3 Web Application 5.4 Correctness Testing 5.5 Performance Comparison 5.6 Duplicate Detection Results	17–32
6	Challenges & Limitations	6.1 Issues Faced During Development 6.2 Solutions Applied 6.3 Limitations of the System	33–34
7	Conclusion & Future Work	7.1 Conclusion 7.2 Future Work 7.3 Skills Learned During Development	35–36
	References	Books, research papers and documentation used	37

List of Figures

1.1 Block diagram of the Question Bank Management System....2
3.1 High-level architecture of QBMS....6
3.2 Class diagram of the Java console client....8
3.3 Flowchart of the Java console application....9
4.1 Aho-Corasick automaton for {he, she, his, hers} (dashed: non-root failure links)....12
4.2 DP matrix for "binry" and "binary" (highlighted: optimal edit path)....14
5.1 Main menu of the console application....17
5.2 KMP keyword search in the console....18
5.3 Aho-Corasick multi-pattern search in the console....19
5.4 Adding a new question and finding it with KMP....19
5.5 Fuzzy search in the console: misspelt query vs. correct query....20
5.6 Similarity check between questions 1 and 5....20
5.7 Duplicate detection at a 70% threshold....20
5.8 Login portal with student and administrator roles....21
5.9 Administrator dashboard: 74 questions, filters and statistics....21
5.10 Question documents after importing the four JSON files....22
5.11 Quick search with highlighted matches in the question table....22
5.12 Global fuzzy search for the misspelt word "deadlok"....23
5.13 Searching Algorithms hub: benchmark of all algorithms on "Binary Search"....24
5.14 KMP view with the computed LPS array and match positions....25
5.15 Rabin-Karp view showing the pattern hash (59 for "deadlock") and matches....25
5.16 Aho-Corasick view with automaton states and failure links....26
5.17 Fuzzy search for "binry serch" ranked by edit distance....26
5.18 Pairwise similarity with word-level comparison....27
5.19 Duplicate detection matrix at a 60% threshold....28
5.20 Generated question paper grouped by unit....28
5.21 Add Question dialog (administrator only)....29
5.22 Student view: read-only access without edit controls....29
5.23 Single-keyword search time over 102,000 questions....30
5.24 Three-keyword search: Aho-Corasick vs. repeated KMP....31

List of Tables

2.1 Functional requirements....4
2.2 Hardware and software requirements....5
3.1 Project data files (corpus)....7
4.1 Source files of the project....10
4.2 LPS array for the pattern "ababaca"....11
4.3 Time and space complexity of the implemented algorithms....16
5.1 Randomised correctness tests against brute-force search....30
5.2 Single-keyword search over 102,000 questions (time in ms)....30
5.3 Pairs flagged at a 60% similarity threshold (74 questions)....31

1. INTRODUCTION

1.1 Overview

A question bank is a structured collection of examination questions, each tagged with information such as subject, unit, difficulty level and the year in which it was asked. When a faculty member sets a question paper, they search the bank by topic, choose a balanced mix of easy, medium and hard questions, and make sure that the same question is not repeated in slightly different words. As the bank grows to hundreds or thousands of entries spread over several files and semesters, doing this by hand becomes slow and error-prone.

The **Question Bank Management System (QBMS)** applies Data Structures and Algorithms to this problem. Every question is stored as a record with an identifier, question text, subject, unit, difficulty and year. Searching and checking are done with string-matching algorithms studied in the DSA-3 course:

- **Exact keyword search** with the Knuth-Morris-Pratt (KMP) algorithm, the Rabin-Karp rolling-hash algorithm and, in the web client, the Boyer-Moore-Horspool algorithm.
- **Multi-keyword search** with the Aho-Corasick automaton, which finds all of a set of keywords in a single pass over each question.
- **Fuzzy (typo-tolerant) search, similarity scoring and duplicate detection** with the Levenshtein edit distance, computed using dynamic programming.

The project is delivered as two clients. The **Java console application** (`Main.java` and five supporting classes) demonstrates the core operations and algorithms through a text menu. The **web application** (`index.html`, `app.js`, `algorithms.js` and `styles.css`) runs entirely in the browser. It adds login roles, multiple question documents, filters, JSON import and export, visual step-by-step views of each algorithm, a side-by-side benchmark and an exam-paper generator. Fig. 1.1 shows the overall flow of information through the system.

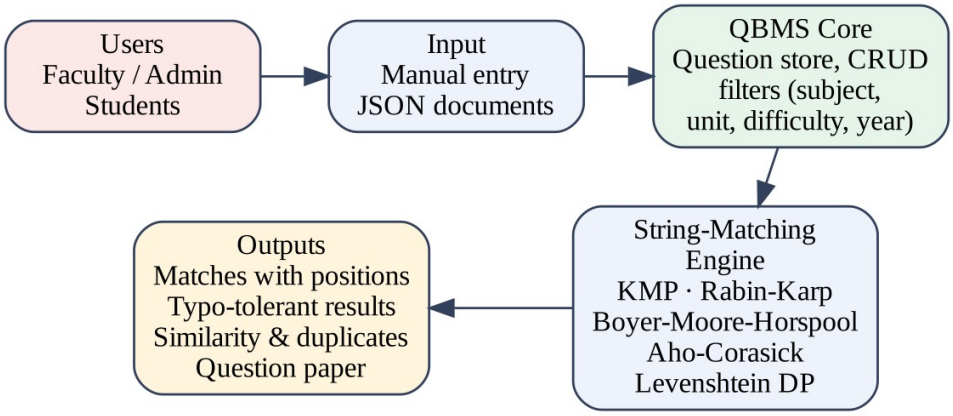


Fig. 1.1 Block diagram of the Question Bank Management System

1.2 Problem Statement

Faculty members preparing examinations must search, filter and de-duplicate large question banks. With manual methods or simple text search this is time-consuming and unreliable, for the following reasons:

- **Slow manual retrieval** – locating every question on a topic such as “deadlock” or “binary search” across several subject files means reading through all of them.
- **Exact-match search fails on small variations** – a misspelt query such as “binry serch” returns nothing with ordinary substring or SQL `LIKE` search.
- **Several keywords need several passes** – checking whether questions mention any of a list of topics usually means repeating a search once per keyword.
- **Duplicates go unnoticed** – the same question reappears in different sets with small changes in wording (for example, an extra clause such as “and analyze Belady's Anomaly”), which lowers the quality of the bank.
- **Unbalanced papers** – without filters on unit and difficulty it is hard to assemble a question paper that covers all units at the right level.

The problem addressed by this project is therefore: *to design and implement a question bank system that stores questions with their metadata and supports fast exact search, multi-keyword search, typo-tolerant search and near-duplicate detection, using efficient string-matching algorithms.*

1.3 Scope of the Project

The scope of the current implementation is as follows:

- Storage of questions with the attributes *id, question text, subject, unit, difficulty (Easy/Medium/Hard)* and *year*. The web client also groups questions into named

documents (question sets) with a title, course code, year and maximum marks.

- Create, read, update and delete (CRUD) operations on questions in both clients, with input validation.
- Exact search (KMP, Rabin-Karp, Boyer-Moore-Horspool), multi-pattern search (Aho-Corasick) and approximate search (Levenshtein) over the question text.
- Pairwise similarity checking between two questions and automatic duplicate detection across the whole bank at a user-chosen threshold.
- Import of question documents from JSON files and export to JSON, Word-compatible HTML and plain text (web client).
- Role-based access in the web client: students can view and search; administrators can also add, edit, delete, import and create documents.
- Generation of a printable question paper grouped by unit (web client).

The following are **outside** the current scope: a server-side database, secure server-side authentication, semantic (meaning-based) similarity, and multi-user editing. These are discussed as future work in Chapter 7.

1.4 Objectives

1. To represent questions and question sets with suitable data structures and support add, update, delete and filter operations.
2. To implement exact pattern matching with KMP and Rabin-Karp (and Boyer-Moore-Horspool in the web client) and report every position where a keyword occurs.
3. To implement the Aho-Corasick automaton so that several keywords can be searched in one pass.
4. To implement fuzzy search based on Levenshtein edit distance so that queries with spelling mistakes still return relevant questions.
5. To compute similarity scores between questions and flag likely duplicates across the bank.
6. To verify the correctness of every algorithm against a brute-force reference and to measure and compare their running times on the project corpus.
7. To provide an easy-to-use interface, both as a console menu and as a web application, that helps faculty prepare balanced question papers.

2. SYSTEM REQUIREMENTS

2.1 Functional Requirements

Table 2.1 lists the functional requirements of QBMS and the client in which each is currently implemented.

Table 2.1 Functional requirements

ID	Requirement	Java console	Web app
FR-01	Add a question with text, subject, unit, difficulty and year; validate every field	Yes	Yes
FR-02	View all questions, edit a question by ID (keep old value on blank input) and delete with confirmation	Yes	Yes
FR-03	Exact keyword search using KMP	Yes	Yes
FR-04	Exact keyword search using Rabin-Karp rolling hash	Yes	Yes
FR-05	Exact keyword search using Boyer-Moore-Horspool	–	Yes
FR-06	Multi-keyword search (comma-separated) using Aho-Corasick, showing which pattern matched where	Yes	Yes
FR-07	Fuzzy search using Levenshtein edit distance with an adjustable threshold	Yes	Yes
FR-08	Similarity score between two selected questions with a duplicate verdict	Yes	Yes
FR-09	Scan the whole bank for duplicate pairs above a user-given threshold	Yes	Yes
FR-10	Filter questions by document, subject, unit and difficulty; quick search with highlighting	–	Yes
FR-11	Login as student (roll number) or administrator (employee ID) with role-based permissions	–	Yes
FR-12	Create question documents; import from JSON; export as JSON, Word-compatible HTML or text	–	Yes
FR-13	Benchmark all search algorithms on the same query and show time and comparisons	–	Yes
FR-14	Generate a printable question paper grouped by unit	–	Yes

2.2 Non-Functional Requirements

- **Performance** – each search should run in time linear in the length of the text for KMP and Aho-Corasick, and should respond instantly on banks of a few thousand questions. (Measured results are given in Section 5.5.)
- **Correctness** – all exact-search algorithms must return the same set of matches as a brute-force search, ignoring letter case.
- **Usability** – menus and forms must reject invalid input with a clear message instead of

crashing; the web client highlights matched text and shows algorithm internals (LPS array, hash values, automaton states, DP matrix) for learning.

- **Portability** – the console application needs only a standard Java Development Kit; the web client needs only a modern browser and runs from a local file, without a server.
- **Maintainability** – each algorithm is a separate class (Java) or a separate object inside `Algorithms` (JavaScript), so it can be tested and replaced independently.
- **Persistence** – the web client keeps documents, questions, the active document, the selected theme and the logged-in user in the browser's `localStorage`, so data survives page reloads.
- **Security** – only administrators may modify the bank. In the current version this rule is enforced in the browser only (see Section 6.3).

2.3 Hardware and Software Requirements

Table 2.2 Hardware and software requirements

Category	Requirement
Processor	Any dual-core x86-64 or ARM processor
Memory	4 GB RAM minimum (8 GB recommended for the IDE and browser together)
Storage	About 50 MB free space for the source code, data files and JDK cache
Operating system	Windows, Linux or macOS
Language and runtime	Java SE (JDK 8 or later; tested with OpenJDK 21) for the console client
Web technologies	HTML5, CSS3 and JavaScript (ES6) in any modern browser such as Chrome, Edge or Firefox
External library	Font Awesome 6.4 icons (loaded from the cdnjs CDN); no other libraries are used
Development tools	Visual Studio Code, Git and GitHub
Data format	JSON files for question documents

3. SYSTEM DESIGN

3.1 High-level Architecture Diagram

QBMS follows a layered architecture (Fig. 3.1). The two clients share the same algorithm layer design: the Java classes `KMP`, `RabinKarp`, `AhoCorasick` and `Levenshtein` have equivalent JavaScript implementations in `algorithms.js`, which also adds Boyer-Moore-Horspool and a benchmarking routine. Each client has its own control logic and its own in-memory data store.

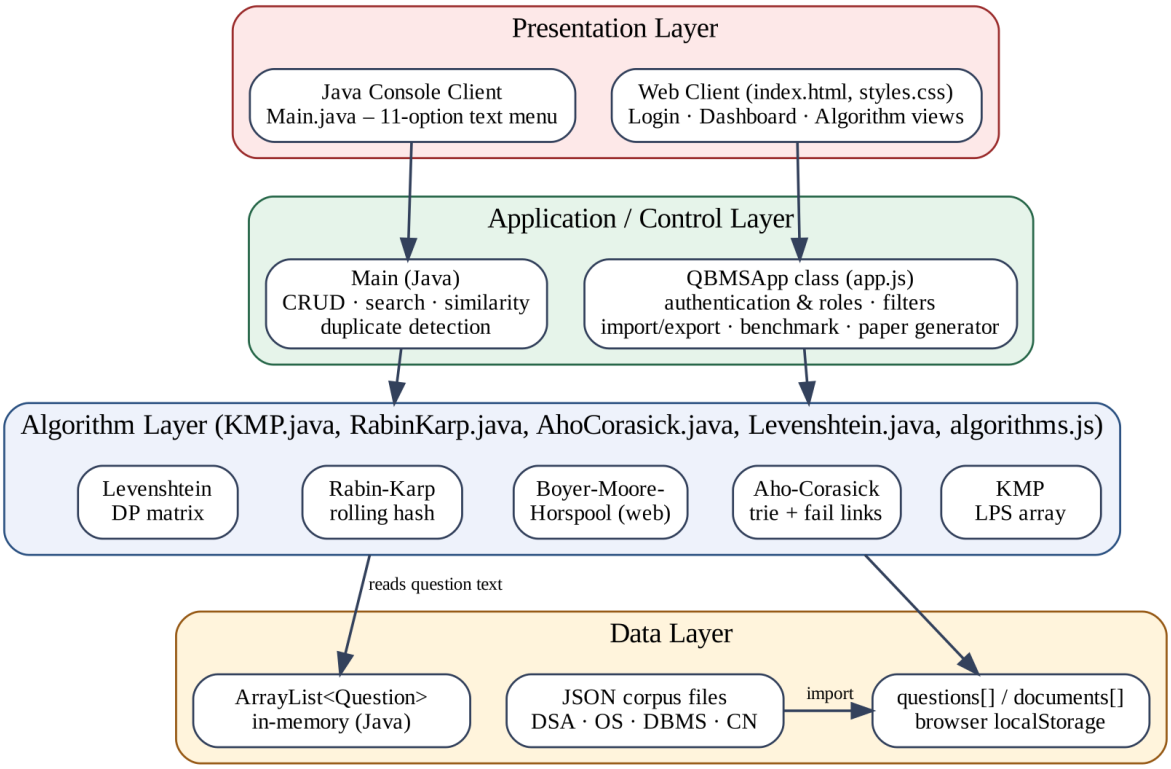


Fig. 3.1 High-level architecture of QBMS

3.2 Description of Each Layer

3.2.1 Presentation Layer

The **Java console client** prints an eleven-option menu (add, view, edit, delete, KMP search, Rabin-Karp search, Aho-Corasick search, fuzzy search, question similarity, duplicate detection and exit) and reads the user's choice with `Scanner.nextLine()`. The **web client** is a single page (`index.html`) with a login overlay, a sidebar for navigation and a set of view panels: Dashboard (question table), Question Documents, Searching Algorithms hub, KMP, Rabin-Karp, Aho-Corasick and Fuzzy search pages, Similarity Check, Duplicate Matrix and Paper Generator. `styles.css` provides light and dark themes.

3.2.2 Application / Control Layer

In Java, the static methods of `Main` read and validate input (`readInt`, `readUnit`, `readDifficulty`, `readYear` and their “or keep” variants for editing), call the algorithm classes and print results. In the web client, the `QBMSApp` class (about 2,900 lines in `app.js`) handles authentication and roles, switching views, filtering and rendering the question table, importing and exporting documents, running the benchmark and generating question papers.

3.2.3 Algorithm Layer

This layer contains the string-matching algorithms described in Chapter 4. All of them convert text and pattern to lower case first, so every search ignores case. The JavaScript versions also return internal data such as the LPS array, every rolling-hash window, the automaton's states and transitions, and the full DP matrix. The web client uses this data to visualise each algorithm step by step.

3.2.4 Data Layer

The Java client keeps questions in an `ArrayList<Question>` and assigns IDs from a counter (`nextId`). It starts with 14 built-in sample questions. The web client keeps two arrays, `documents` and `questions`, where each question refers to its document through `docId`. It starts with 4 documents and 23 questions and stores them in `localStorage`. The project corpus is a set of four JSON files that can be imported into the web client (Table 3.1).

Table 3.1 Project data files (corpus)

File	Subject	Questions	Easy / Medium / Hard
sample_questions.json	DSA	12	3 / 4 / 5
sample_questions_os.json	Operating Systems	13	4 / 5 / 4
sample_questions_dbms.json	DBMS	13	3 / 5 / 5
sample_questions_cn.json	Computer Networks	13	3 / 6 / 4
Total		**51**	**13 / 20 / 18**

Each file has a `documentMetadata` object (title, subject, course code, year, maximum marks and description) and a `questions` array. The structure is shown below:

```
{
  "documentMetadata": {
    "title": "DSA Question Bank 2026 - Comprehensive Set",
    "subject": "DSA", "code": "23CS201", "year": 2026, "maxMarks": 100,
    "description": "Comprehensive question set covering Arrays, ..."
  },
  "questions": [
    { "question": "Explain the working and complexity analysis of
                    Binary Search with an example.",
      "subject": "DSA", "unit": 1, "difficulty": "Easy", "year": 2026 },
    ...
  ]
}
```

Listing 3.1 Structure of a question document (sample_questions.json)

3.3 Class Design

Fig. 3.2 shows the classes of the Java console client. `Question` is a plain data class. `Main` holds the list of questions and the menu logic, and uses the four algorithm classes. These classes have only static methods, except `AhoCorasick`, which must first be built from a set of patterns.

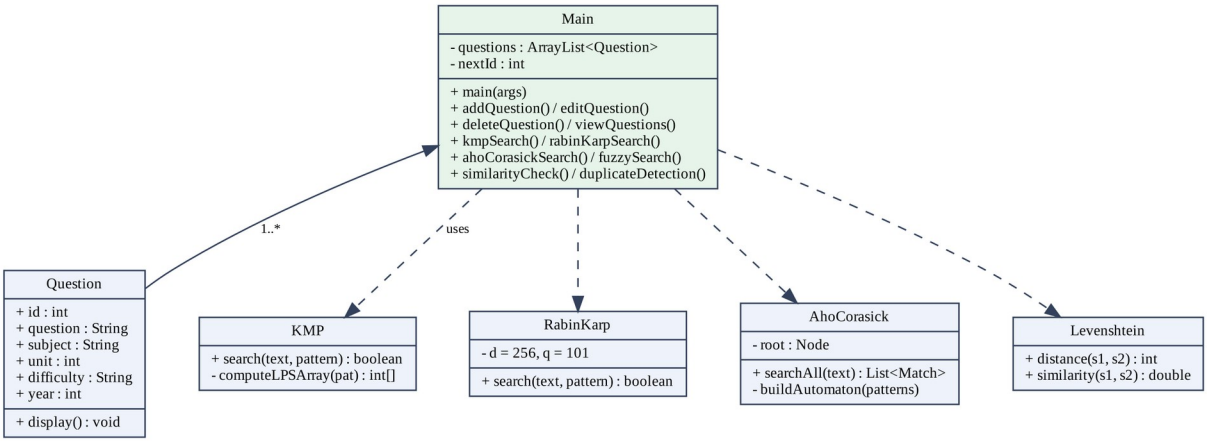


Fig. 3.2 Class diagram of the Java console client

3.4 Control Flow

Fig. 3.3 shows the control flow of the console application. After loading the sample questions, the program shows the menu repeatedly until the user chooses option 11. Input that is not a number from 1 to 11 is rejected with a message, so the program does not crash on bad input.

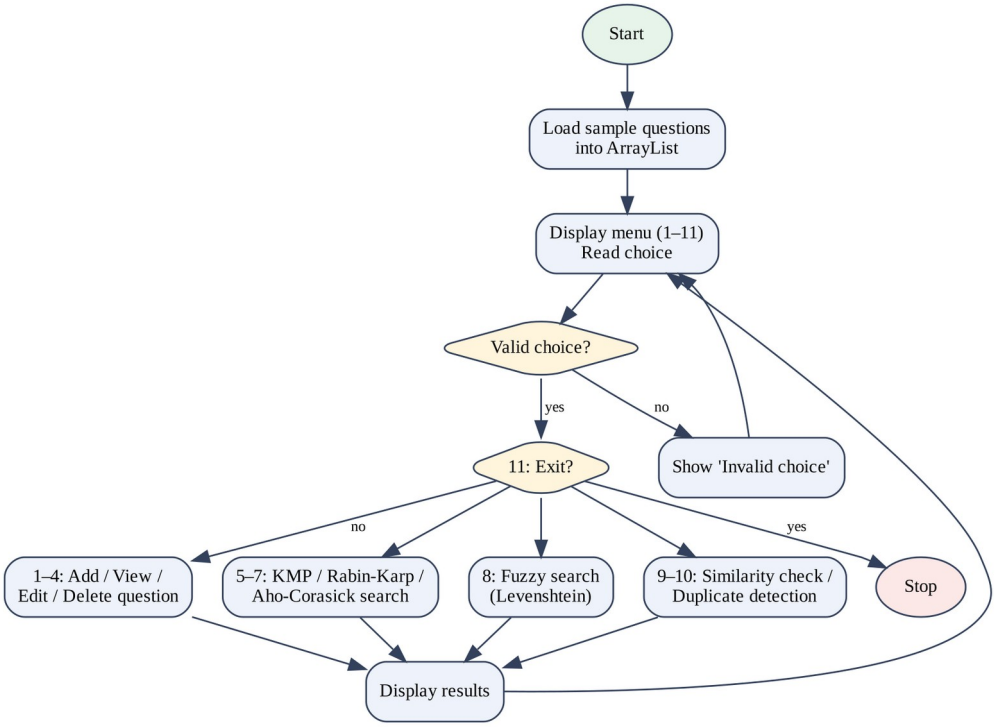


Fig. 3.3 Flowchart of the Java console application

4. IMPLEMENTATION

This chapter describes how each module is implemented. Table 4.1 summarises the source files.

Table 4.1 Source files of the project

File	Lines	Purpose
Main.java	518	Console menu, CRUD operations, search handlers, similarity and duplicate detection, input validation
Question.java	27	Question record and display()
KMP.java	61	Knuth-Morris-Pratt search with LPS array
RabinKarp.java	66	Rabin-Karp search with rolling hash (d = 256, q = 101)
AhoCorasick.java	152	Trie, failure links and output links; searchAll() returns every match
Levenshtein.java	50	Edit distance (DP) and percentage similarity
algorithms.js	639	JavaScript versions of all algorithms plus Boyer-Moore-Horspool, benchmark and word diff
app.js	2,870	QBMSApp class: authentication, views, filters, import/export, paper generator
index.html / styles.css	1,435 / 2,749	Page structure and light/dark styling of the web client

4.1 Module-wise Implementation

4.1.1 Question Store and CRUD Module

In Java, `addQuestion()` reads each field with a validating helper. `readUnit` accepts only positive integers, `readDifficulty` accepts only Easy, Medium or Hard in any letter case and stores it in title case, and `readYear` accepts 1900–2100. Every value is read with `nextLine()` and parsed afterwards, which avoids the common Scanner problem of a stray newline being left in the buffer. `editQuestion()` shows the current value of each field in brackets and keeps it if the user presses Enter. `deleteQuestion()` displays the question and asks for Y/N confirmation. `findQuestion(id)` scans the list for the matching ID.

In the web client, the add and edit form is a modal dialog, and only administrators can see the Add, Edit, Delete, Import and Create Document buttons. Every change is saved immediately to `localStorage` through `saveQuestions()` and `saveDocuments()`.

4.1.2 KMP (Knuth-Morris-Pratt) Search

Naive search re-examines text characters after a mismatch. KMP avoids this by first computing the **longest proper prefix which is also a suffix (LPS)** for every prefix of the pattern. When a mismatch occurs after *j* matched characters, the search continues from `lps[j-1]` without moving

back in the text. Preprocessing takes O(m) time and the search takes O(n), where *m* is the pattern length and *n* the text length.

```
private static int[] computeLPSArray(String pat) {
    int M = pat.length();  int[] lps = new int[M];
    int len = 0, i = 1;       // len = previous longest prefix-suffix
    while (i < M) {
        if (pat.charAt(i) == pat.charAt(len)) { lps[i++] = ++len; }
        else if (len != 0) { len = lps[len - 1]; }  // fall back
        else { lps[i++] = 0; }
    }
    return lps;
}
```

Listing 4.1 LPS computation in KMP.java (condensed)

Table 4.2 LPS array for the pattern "ababaca"

Index	0	1	2	3	4	5	6
Character	a	b	a	b	a	c	a
LPS value	0	0	1	2	3	0	1

The Java method `KMP.search(text, pattern)` returns `true` as soon as the first occurrence is found, because the console only needs to know whether a question matches. The JavaScript version continues after each match by setting `j = lps[j-1]`, so it reports **every** starting index along with a trace of each comparison. The web client uses this trace to draw the LPS array and highlight the matched text.

4.1.3 Rabin-Karp Search

Rabin-Karp compares a numeric hash of the pattern with a hash of each m-character window of the text. The hash is a base-*d* polynomial taken modulo a prime *q*. The next window's hash is calculated from the previous one in constant time by removing the leading character and adding the next one:

```
// h = d^(M-1) mod q
t = (d * (t - txt.charAt(i) * h) + txt.charAt(i + M)) % q;
if (t < 0) t = t + q;          // keep the hash non-negative
```

Listing 4.2 Rolling-hash update in RabinKarp.java

When the hashes are equal, the window is compared with the pattern character by character, because different strings can share a hash value (a *spurious hit*). The project uses $d = 256$ and $q = 101$. For example, the pattern “deadlock” has hash 59. The average running time is O(n + m); the worst case is O(nm) when many spurious hits occur. Section 5.5 shows how the small modulus affects the number of spurious hits.

4.1.4 Aho-Corasick Multi-Pattern Search

When a user searches for several keywords at once (for example “binary, tree, deadlock”), running

KMP once per keyword reads every question once per keyword. Aho-Corasick builds a single automaton from all the keywords and finds every occurrence of every keyword in **one pass** over the text, in O(n + total pattern length + number of matches) time. It is built in three steps:

- **Trie construction** – each pattern (in lower case) is inserted into a trie whose nodes store their children in a `HashMap<Character, Node>`. The node at the end of a pattern stores that pattern in its `output` list.
- **Failure links** – a breadth-first traversal sets, for each node, a `fail` pointer to the node for the longest proper suffix of its string that is also a path in the trie. Nodes at depth 1 point to the root.
- **Output links** – while the failure links are built, each node also copies the outputs of its failure node. A node can therefore report a short pattern that ends inside a longer one, such as "he" inside "she".

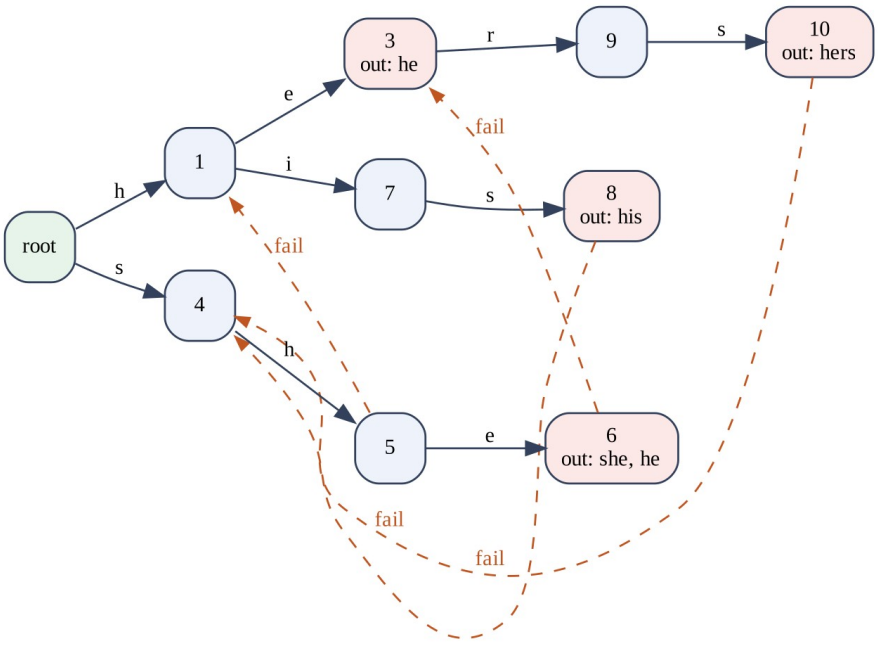


Fig. 4.1 Aho-Corasick automaton for {he, she, his, hers} (dashed: non-root failure links)

```
while (!queue.isEmpty()) {                      // BFS over the trie
    Node curr = queue.poll();
    for (Map.Entry<Character, Node> e : curr.next.entrySet()) {
        char c = e.getKey();  Node child = e.getValue();
        Node f = curr.fail;
        while (f != null && !f.next.containsKey(c)) f = f.fail;
        child.fail = (f != null && f.next.containsKey(c))
                     ? f.next.get(c) : root;
        for (String out : child.fail.output)          // inherit outputs
            if (!child.output.contains(out)) child.output.add(out);
        queue.add(child);
    }
}
```

Listing 4.3 Failure-link construction in AhoCorasick.java (condensed)

During search, the automaton reads one character at a time, follows failure links on a mismatch and, at every node with outputs, records a `Match(pattern, startIndex, endIndex)`. The console prints each matching question together with its matched patterns and positions, for example “binary @0, tree @22”. The web client also displays the states of the trie, its failure links and the output dictionary.

4.1.5 Boyer-Moore-Horspool Search (web client)

Boyer-Moore-Horspool compares the pattern with the text from right to left. Before searching, it builds a *bad-character shift table*: for each character in the pattern except the last, the table stores how far that character is from the end of the pattern. After each comparison, the pattern is shifted by the table value of the text character under the last pattern position, or by the full pattern length if that character does not occur in the pattern. Long patterns can therefore skip large parts of the text, giving a best case of O(n/m); the worst case is O(nm). The web client counts character comparisons so that this algorithm can be compared with the others.

4.1.6 Fuzzy Search with Levenshtein Distance

The Levenshtein distance between two strings is the minimum number of single-character insertions, deletions and substitutions needed to turn one into the other. It is computed with dynamic programming on an (n+1) × (m+1) table:

```
dp[i][0] = i;   dp[0][j] = j;
dp[i][j] = dp[i-1][j-1]                    if s1[i-1] == s2[j-1]
dp[i][j] = 1 + min(dp[i-1][j-1],           (substitution)
                   dp[i-1][j],             (deletion)
                   dp[i][j-1])             (insertion)  otherwise
similarity(%) = (1 - distance / max(len1, len2)) * 100
```

Listing 4.4 Recurrence used in Levenshtein.java

Edit distance("binry", "binary") = 1

	ε	b	i	n	a	r	y
ε	0	1	2	3	4	5	6
b	1	0	1	2	3	4	5
i	2	1	0	1	2	3	4
n	3	2	1	0	1	2	3
r	4	3	2	1	1	1	2
y	5	4	3	2	2	2	1

Fig. 4.2 DP matrix for "binry" and "binary" (highlighted: optimal edit path)

The two clients use the distance in different ways. The **Java** fuzzy search compares the whole query with the whole question, and shows a question if the distance is at most 5 or if the question contains the query as a substring. The **web** client uses `bestSubstringDistance()`, which slides windows of length (query length ± 3) across the question and keeps the smallest distance. Because this compares the query with the closest *part* of the question, a misspelt query such as “binry serch” is matched correctly. Section 5.3 compares the two approaches.

4.1.7 Similarity Check and Duplicate Detection

`similarityCheck()` lets the user pick two questions by ID or by keyword (when more than one question matches, it lists them and asks for an ID) and prints their Levenshtein similarity. Pairs with a similarity of 80% or more are labelled *Possible Duplicate*. `duplicateDetection()` compares every pair of questions, which is N(N−1)/2 comparisons, and reports every pair at or above a threshold entered by the user. The web version also shows a word-level comparison (`getDiffSummary`), with words common to both questions highlighted.

4.2 Front-end Logic (UI Rendering and State Management)

The web client is written without a framework. A single `QBMSApp` object (exposed as `window.app`) holds all application state: `documents`, `questions`, `activeDocId`, `currentView`, `currentUser`, `selectedLoginRole` and the on/off setting for fuzzy table search. The main techniques are:

- **View switching** – each sidebar item carries a `data-view` attribute. Clicking it marks the matching `section.view-panel` as active and calls that view's render function.
- **Template-literal rendering** – tables, cards and result lists are built as HTML strings and

assigned to `innerHTML`. Question text is passed through `escapeHTML()` and the search keyword through `escapeRegExp()` before matches are wrapped in `<mark>` tags.

- **Persistence** – `loadData()` reads `qbms_documents`, `qbms_questions`, `qbms_active_doc`, `qbms_user` and `qbms_theme` from `localStorage`. If they are missing or unreadable, it falls back to the built-in sample data.
- **Role-based rendering** – `applyAuthState()` shows or hides administrator controls. Student roll numbers must match `^25200\d{5}$` and administrator IDs must start with `EMP`.
- **Debounced live search** – the global fuzzy search box in the header waits briefly after the user stops typing and then shows the closest questions, along with their edit distances, in a drop-down list.

4.3 API Integration

QBMS does not call any external web API or server. All processing is done locally, so the system works offline after the page loads (only the icon font is loaded from a CDN). The web client uses the following **browser APIs**:

- **Web Storage API** (`localStorage`) – stores documents, questions, the active document, the theme and the session.
- **File API** (`FileReader.readAsText`) – reads a JSON file chosen by the user for import.
- **JSON API** (`JSON.parse`, `JSON.stringify`) – parses imported documents and serialises data for storage and export.
- **Blob and object URLs** – create downloadable JSON, Word-compatible HTML and plain-text files when a document is exported.
- **High Resolution Time API** (`performance.now()`) – measures the running time of each algorithm in the benchmark hub.

4.4 Import, Export and Paper Generator

`handleImportExecution()` accepts two formats: a structured document with `documentMetadata` and `questions` (the format of the project's JSON files), or a plain array of questions or question strings. Imported questions receive new sequential IDs and are attached to a newly created document. Export writes the active document as JSON, as an HTML file that opens in Microsoft Word, or as text. The paper generator groups the questions in the selected scope by unit and assigns marks by difficulty, producing a printable paper with a title, maximum marks, subject and duration.

4.5 Complexity Summary

Table 4.3 Time and space complexity of the implemented algorithms

Algorithm	Preprocessing	Search time	Extra space
KMP	O(m)	O(n)	O(m)
Rabin-Karp	O(m)	O(n + m) average, O(nm) worst	O(1)
Boyer-Moore-Horspool	O(m + σ)	O(n/m) best, O(nm) worst	O(σ)
Aho-Corasick (k patterns, total length M)	O(M)	O(n + z), z = number of matches	O(M)
Levenshtein distance	–	O(nm)	O(nm)
Duplicate detection (N questions, length L)	–	$O(N^2 \cdot L^2)$	$O(L^2)$

4.6 How to Run

```
# Java console client
javac *.java
java Main

# Web client: open index.html in a browser (no server needed).
# Log in as Student (e.g. 2520040001) or Admin (e.g. EMP1001),
# then use Import Document to load the sample_questions*.json files.
```

Listing 4.5 Compiling and running the project

5. RESULTS

5.1 Test Environment and Dataset

The results in this chapter were produced on a Linux x86-64 machine with OpenJDK 21 and Chromium. The console runs use the 14 built-in questions in `Main.java`. The web runs use the 23 built-in questions plus the 51 questions imported from the four JSON files, which gives **74 questions in 8 documents**. For the timing experiments, the 51-question corpus was repeated 2,000 times to make **102,000 questions (12.1 million characters)**, so that the times are large enough to measure reliably.

5.2 Java Console Application

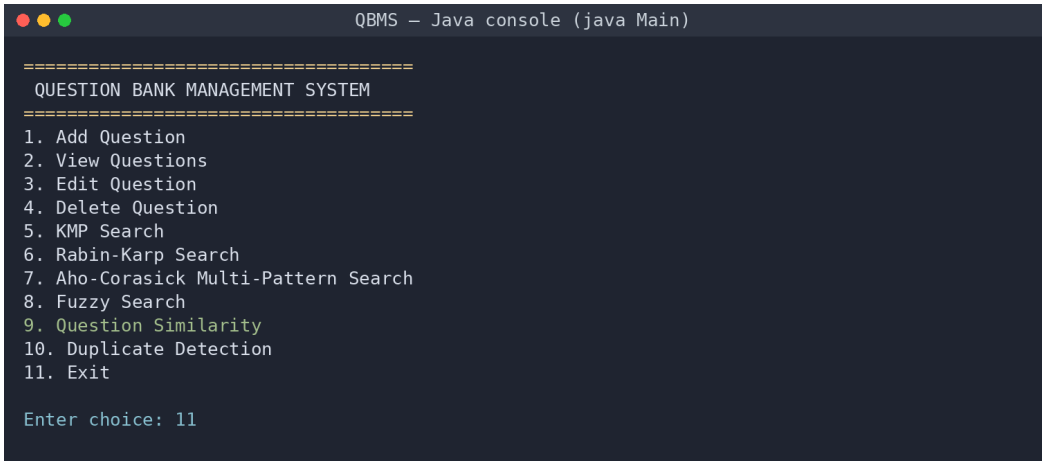


Fig. 5.1 Main menu of the console application

Fig. 5.2 shows a KMP search for “binary search”. The search ignores letter case, so it matches “Binary Search” in questions 1, 2 and 5. Question 2 matches because “Binary Search Tree” contains the keyword.

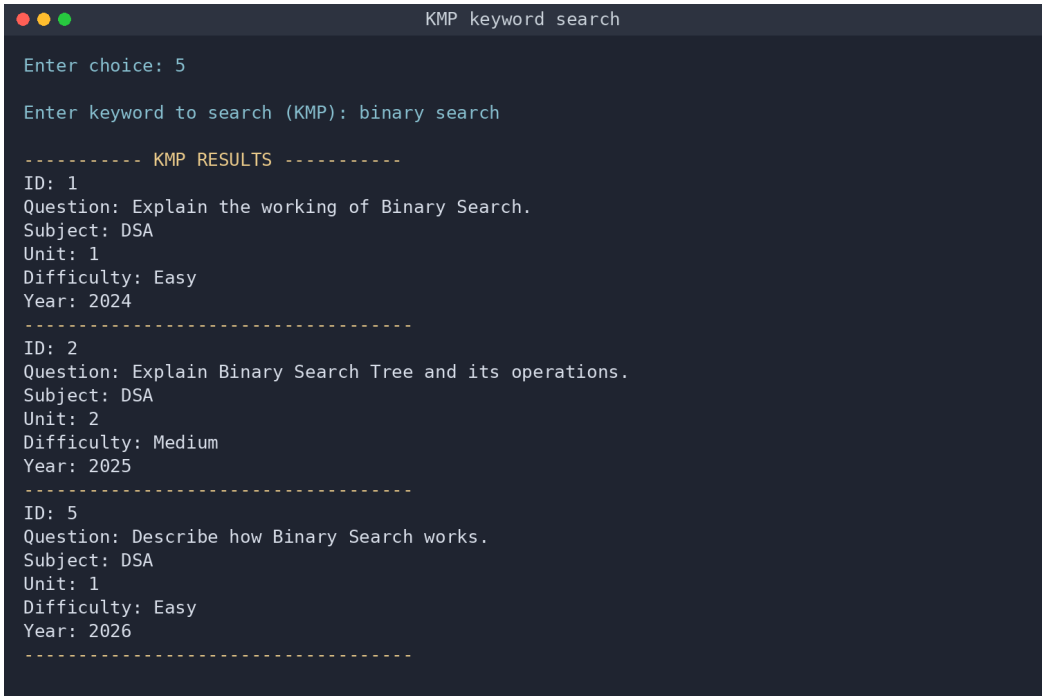


Fig. 5.2 KMP keyword search in the console

Fig. 5.3 shows an Aho-Corasick search for three keywords at once. For each matching question, the console prints which patterns matched and where. For example, question 2 shows “binary @0, tree @22”.

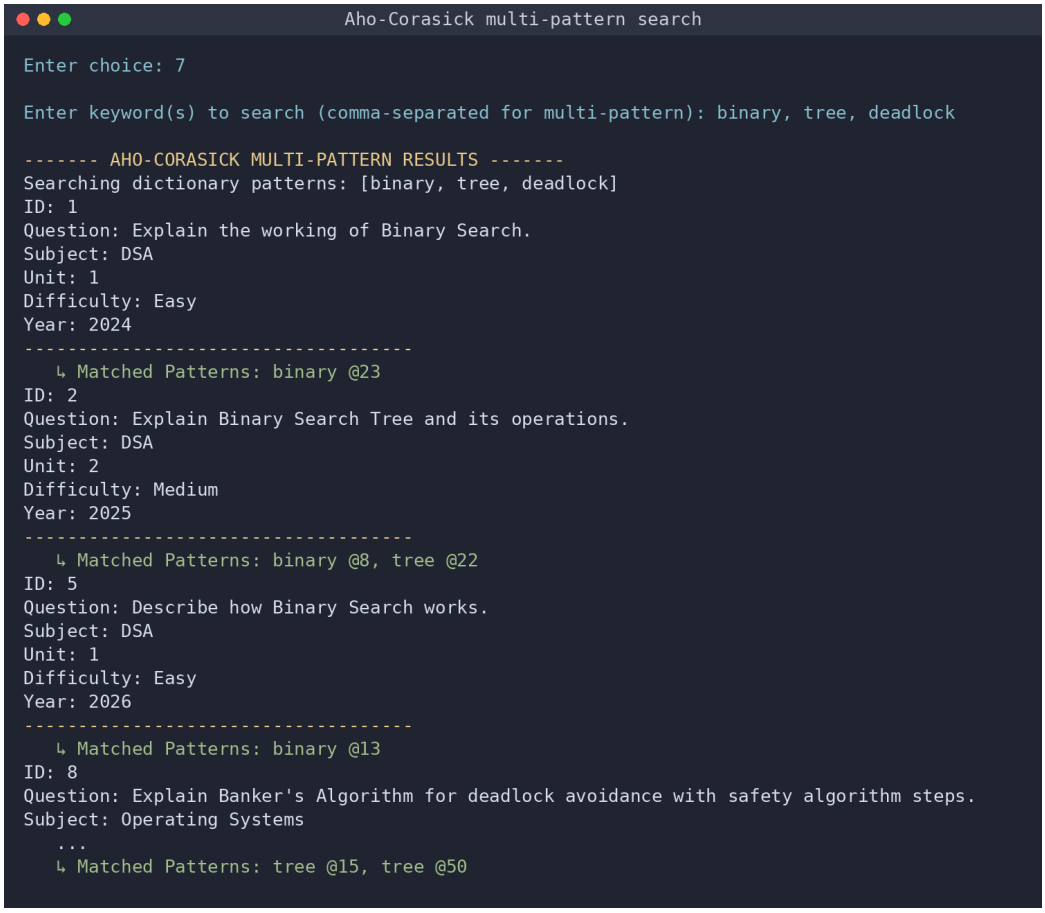


Fig. 5.3 Aho-Corasick multi-pattern search in the console

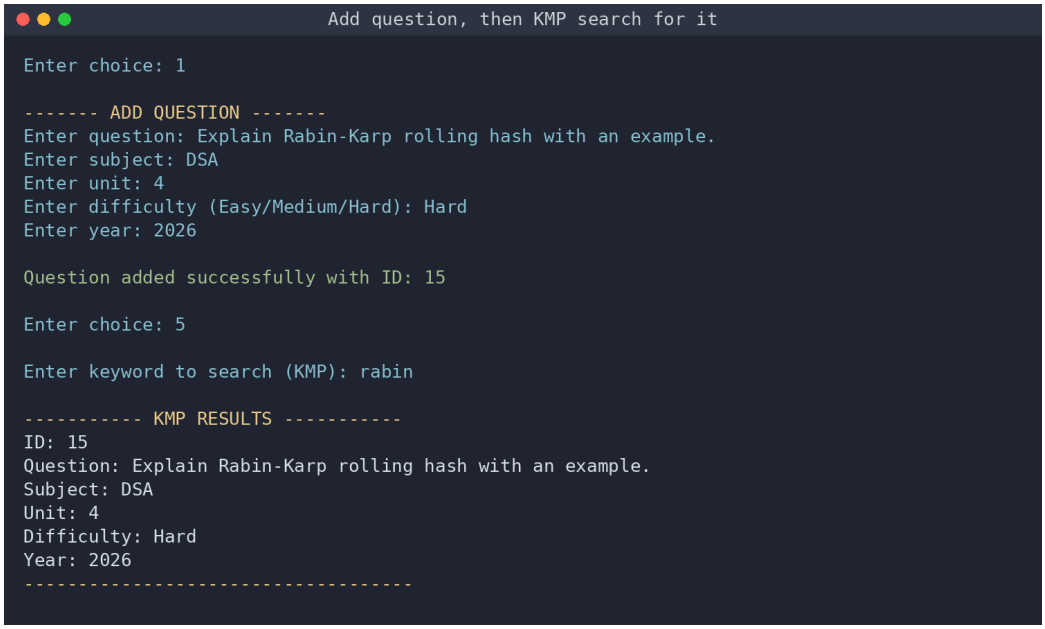


Fig. 5.4 Adding a new question and finding it with KMP

Fig. 5.5 shows a limitation of the console's fuzzy search. The misspelt query “binary serch” returns no results, because its distance to every *whole* question is far above 5. The correctly spelt query works only because of the substring check. Fig. 5.6 and Fig. 5.7 show the similarity checker and the duplicate scan.

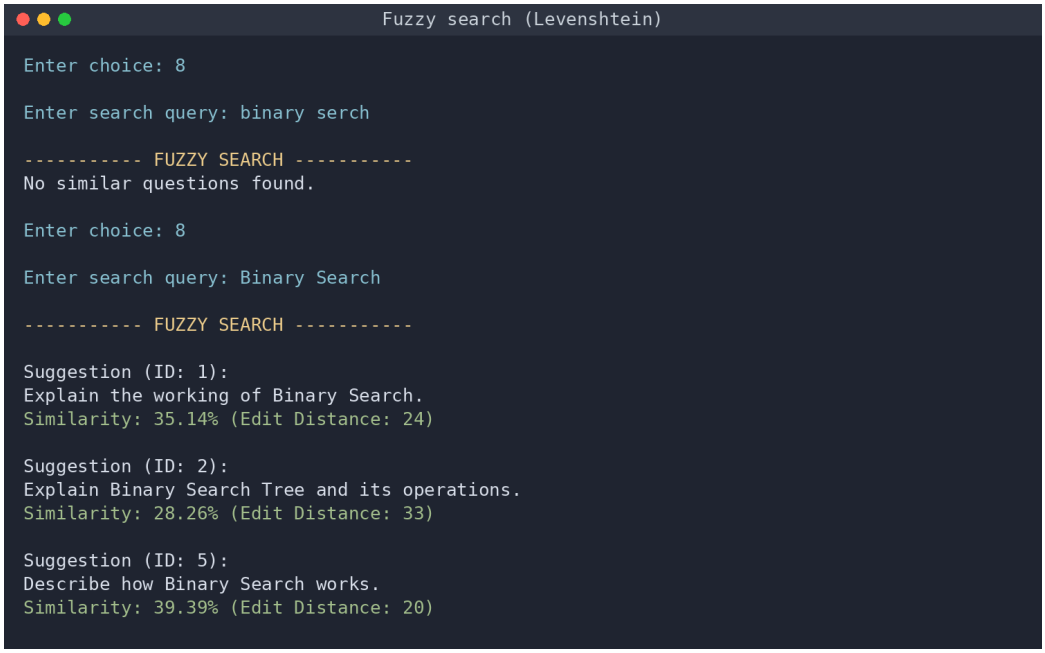


Fig. 5.5 Fuzzy search in the console: misspelt query vs. correct query

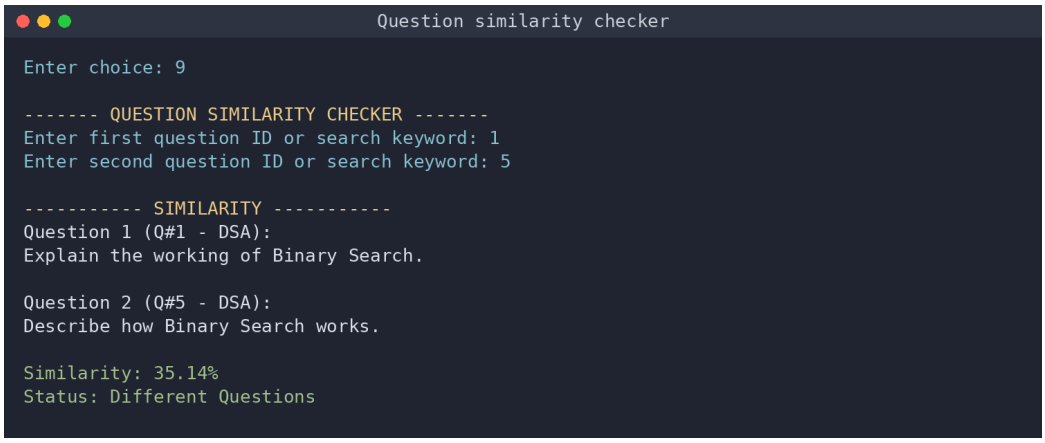


Fig. 5.6 Similarity check between questions 1 and 5

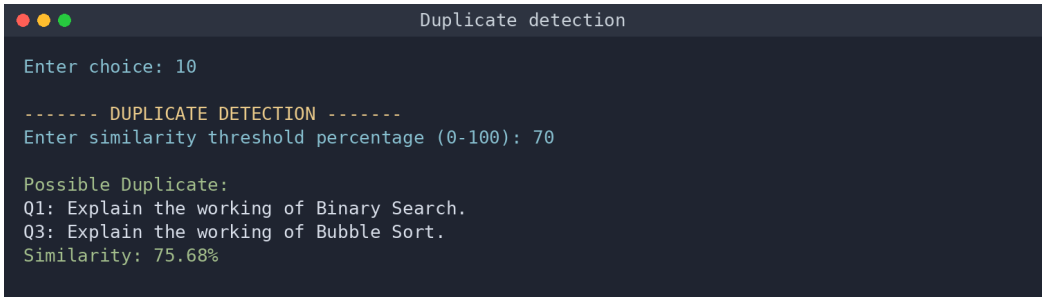


Fig. 5.7 Duplicate detection at a 70% threshold

5.3 Web Application

5.3.1 Login, Dashboard and Documents

The web client opens with a login portal (Fig. 5.8). A student signs in with a 10-digit roll number beginning with 25200, and an administrator signs in with an employee ID beginning with EMP.

Sample accounts are listed on the page for demonstration. After an administrator logs in and imports the four JSON files, the dashboard (Fig. 5.9) shows 74 questions from 4 subjects, the number of duplicates found, and the Easy/Medium/Hard split (19 / 27 / 28). The Documents view (Fig. 5.10) lists each question set with its course code, question count and marks, and has buttons to make it the active set or export it.

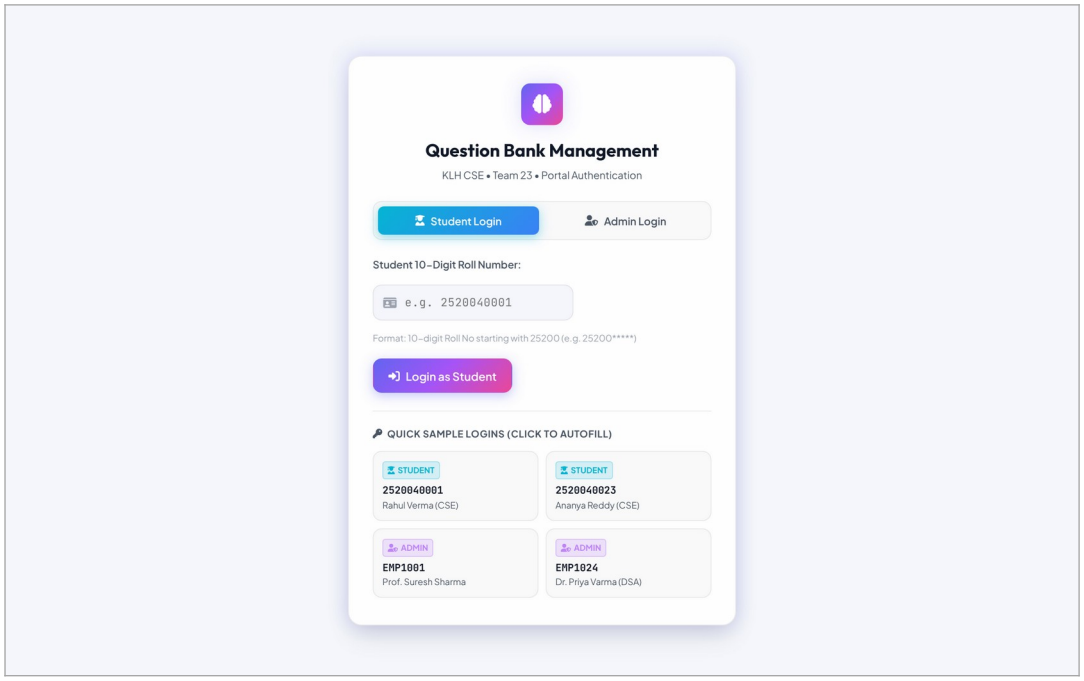


Fig. 5.8 Login portal with student and administrator roles

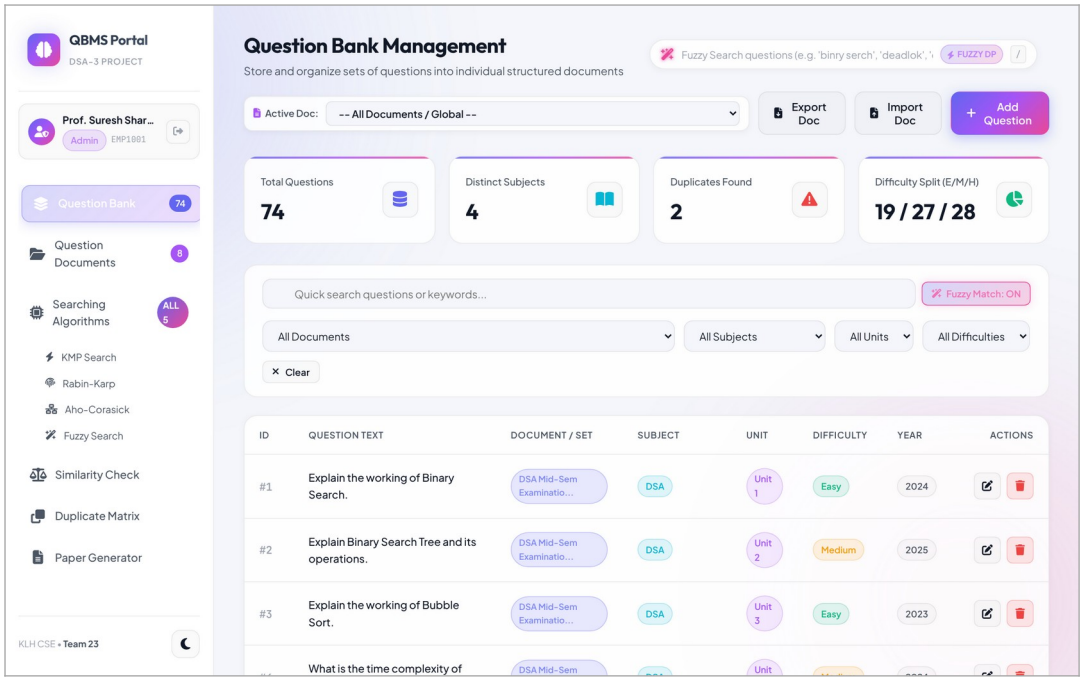


Fig. 5.9 Administrator dashboard: 74 questions, filters and statistics

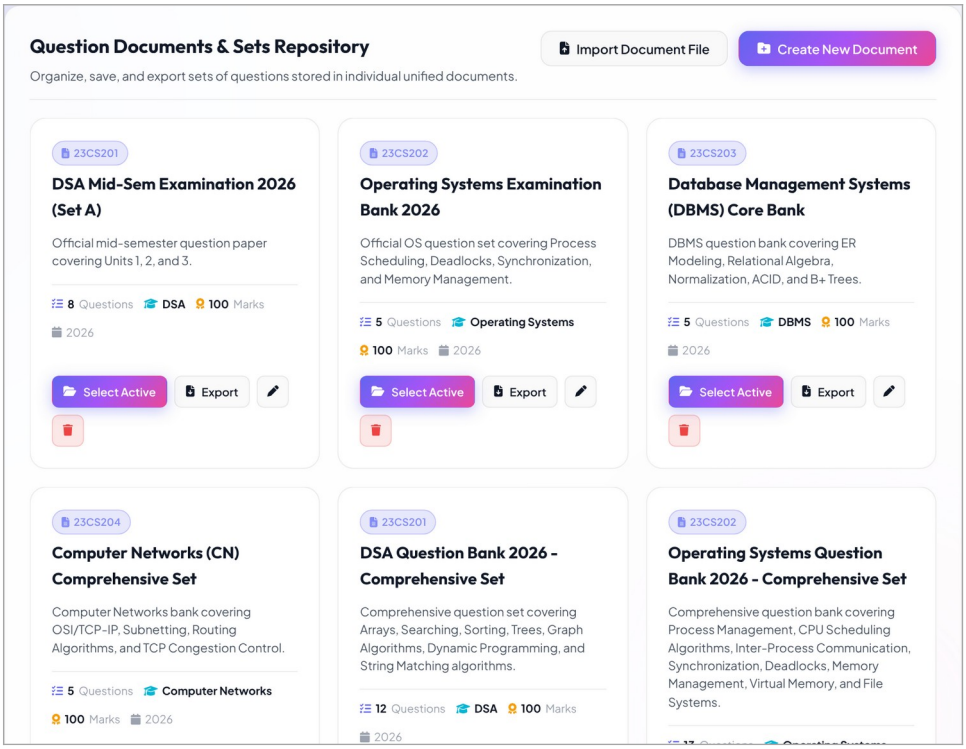


Fig. 5.10 Question documents after importing the four JSON files

5.3.2 Quick Search and Global Fuzzy Search

Typing in the table's quick-search box filters the questions as the user types and highlights the matched text (Fig. 5.11). The header search box performs live fuzzy search over the whole bank. In Fig. 5.12, the misspelt query “deadlok” finds the deadlock questions at edit distance 1, even though no question contains that exact string.

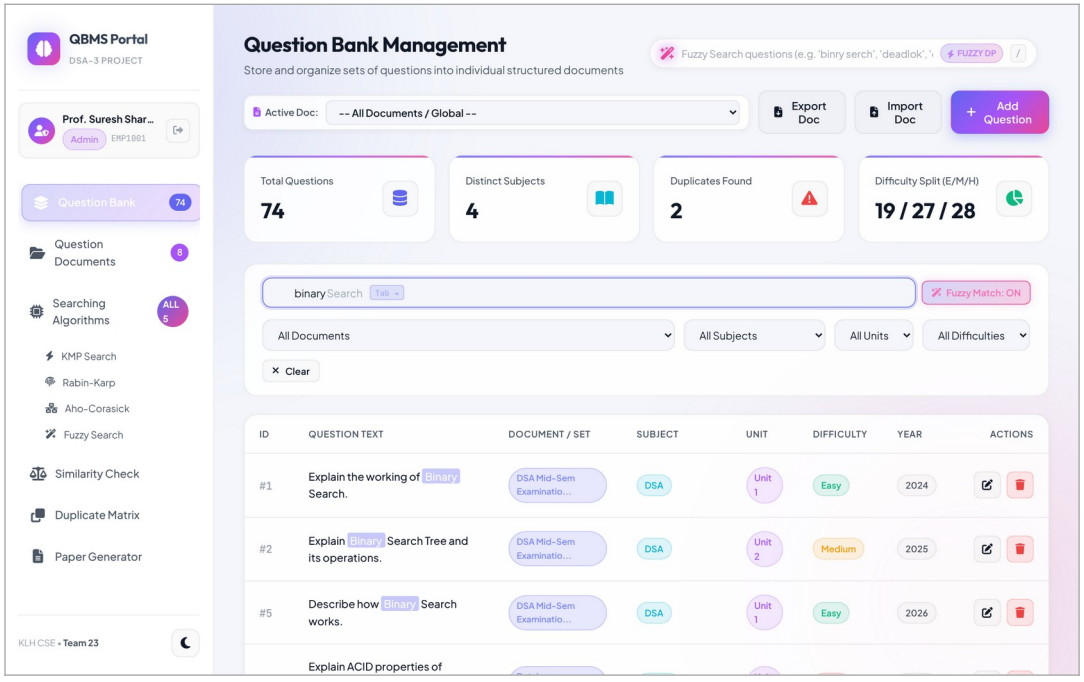


Fig. 5.11 Quick search with highlighted matches in the question table

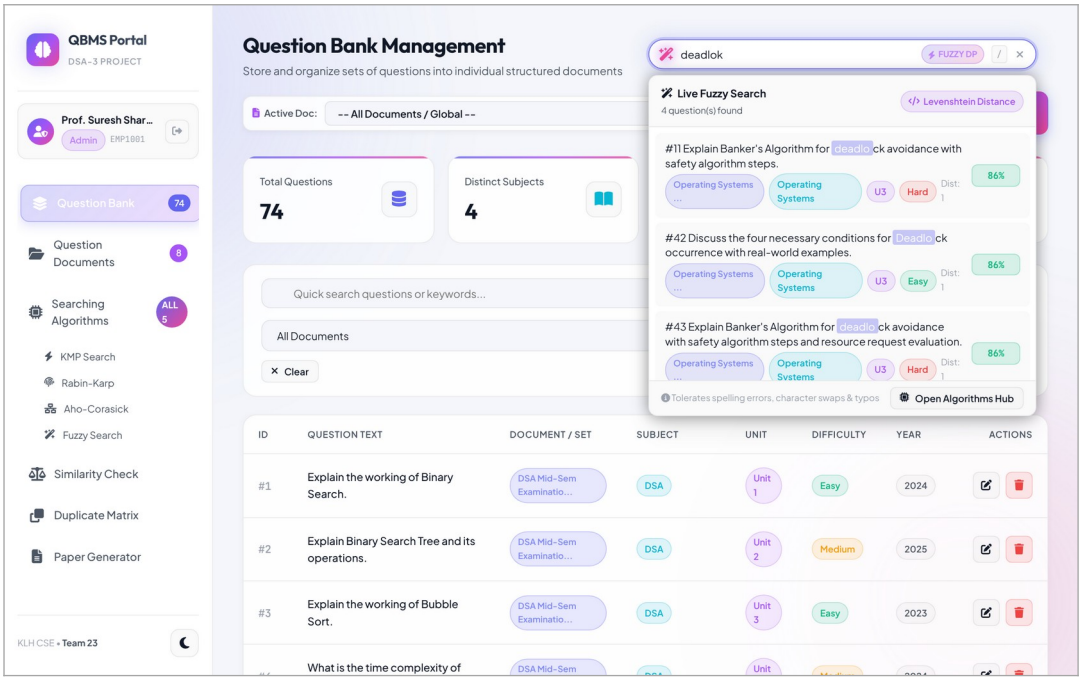


Fig. 5.12 Global fuzzy search for the misspelt word "deadlok"

5.3.3 Searching Algorithms Hub

The hub runs every algorithm on the same query and scope and shows the time, number of matching questions and number of character comparisons for each (Fig. 5.13). For “Binary Search” over all 74 questions, KMP, Rabin-Karp, Aho-Corasick and Boyer-Moore-Horspool each found the same 7 questions. Boyer-Moore-Horspool needed only **979** character comparisons, compared with **7,641** for KMP, because its bad-character table lets it skip ahead by up to the full pattern length. The fuzzy search found 8 questions, one more than the exact methods, because it also accepts approximate matches.

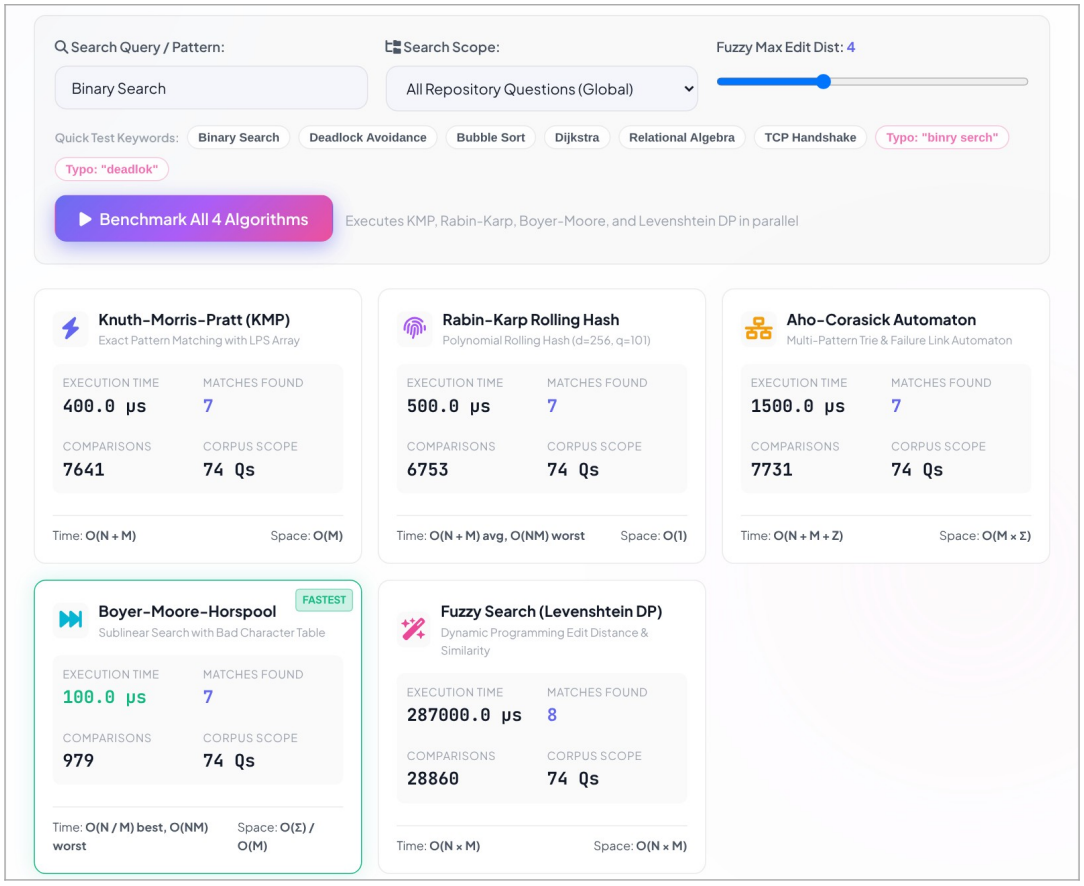


Fig. 5.13 Searching Algorithms hub: benchmark of all algorithms on "Binary Search"

5.3.4 Algorithm Views

Each algorithm has its own page that shows how it works. The KMP view (Fig. 5.14) prints the LPS array for the entered pattern under each character and lists every match. The Rabin-Karp view (Fig. 5.15) shows the base, modulus and pattern hash (59 for “deadlock”) and the number of hash matches per question.

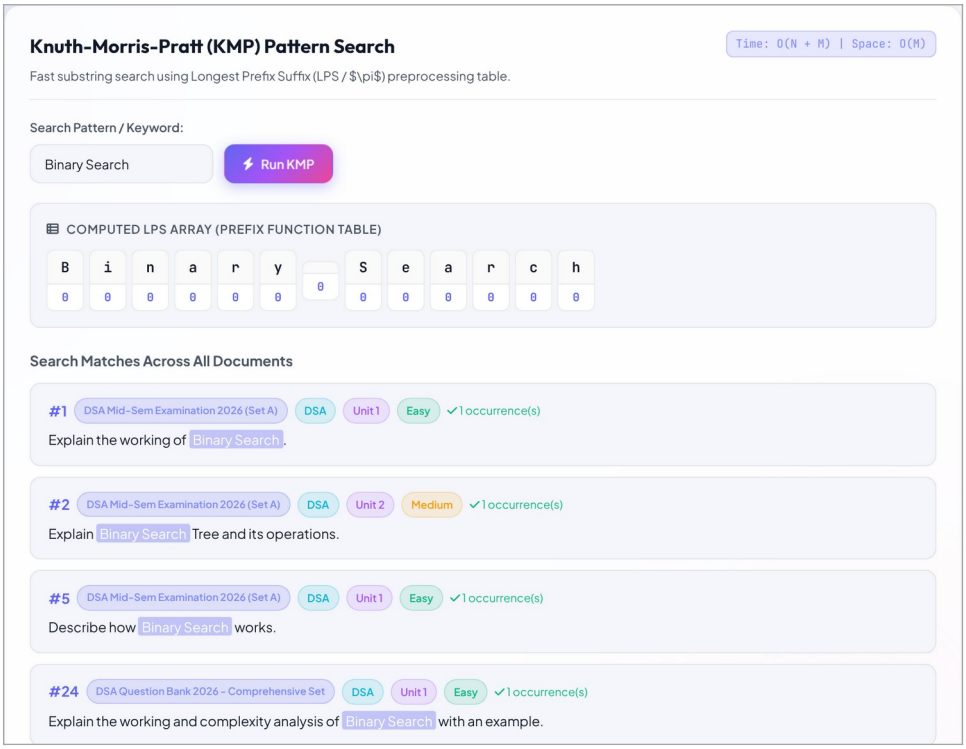


Fig. 5.14 KMP view with the computed LPS array and match positions

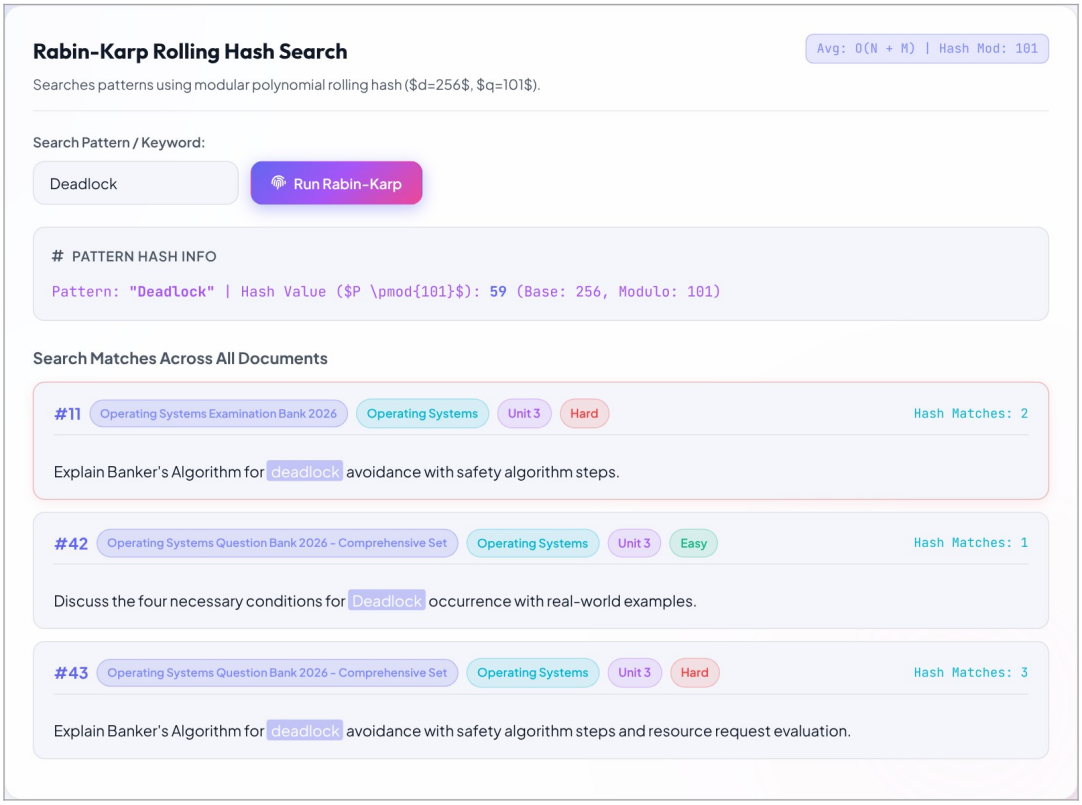


Fig. 5.15 Rabin-Karp view showing the pattern hash (59 for "deadlock") and matches

The Aho-Corasick view (Fig. 5.16) lists every state of the automaton built from the comma-separated keywords, with its depth, transitions, failure link and output set. The fuzzy search view (Fig. 5.17) ranks questions by their best substring edit distance. Searching for “binry serch” returns

7 questions at distance 2 or less, each showing the closest matching text (“Binary Search”), because the web client compares the query with the closest part of each question rather than with the whole question.

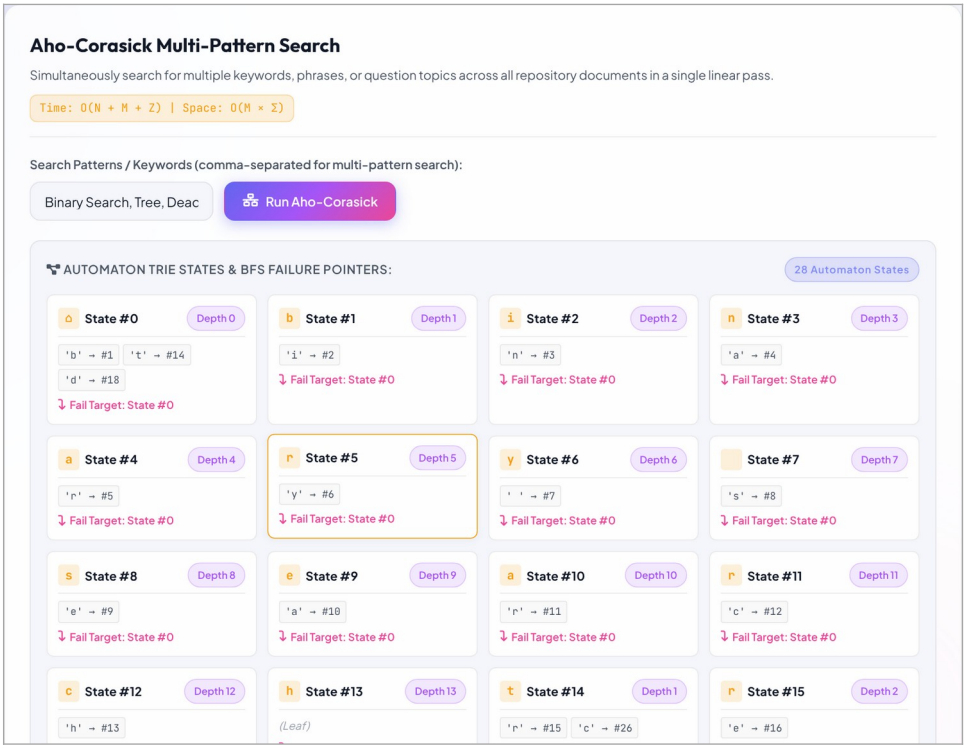


Fig. 5.16 Aho-Corasick view with automaton states and failure links

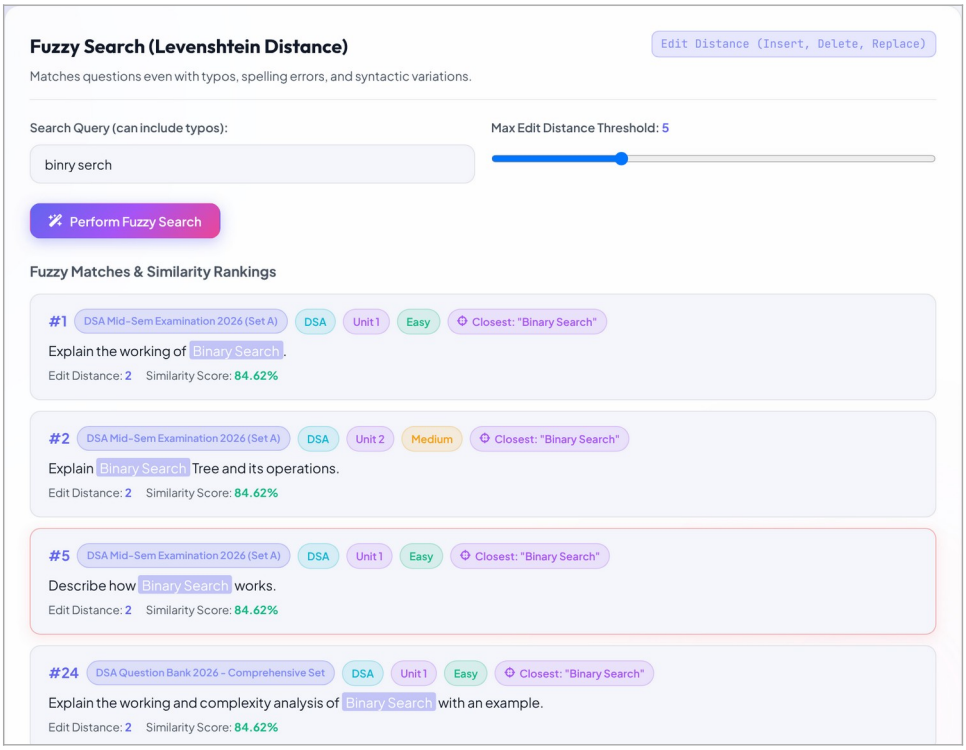


Fig. 5.17 Fuzzy search for "binry serch" ranked by edit distance

5.3.5 Similarity, Duplicates and Paper Generation

The similarity checker (Fig. 5.18) compares any two questions, or any two texts typed by the user, and shows the percentage, the edit distance and the words the two have in common. The duplicate matrix (Fig. 5.19) scans all pairs above the chosen threshold; its results are analysed in Section 5.6. The paper generator (Fig. 5.20) produces a printable paper with questions grouped by unit.

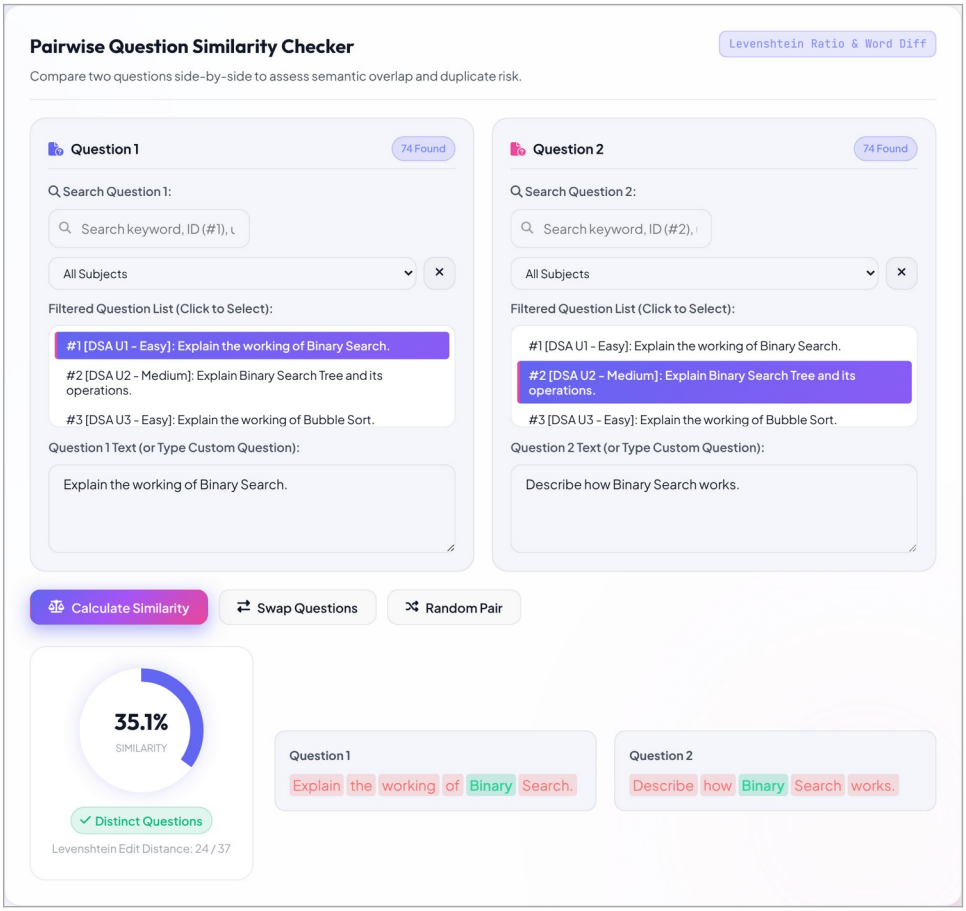


Fig. 5.18 Pairwise similarity with word-level comparison

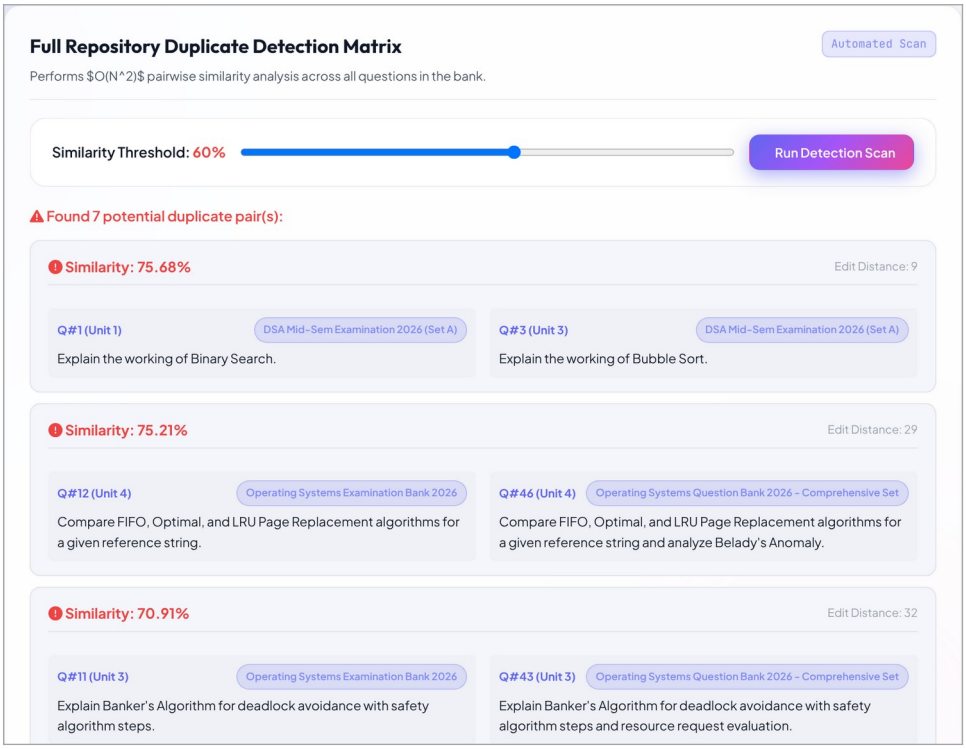


Fig. 5.19 Duplicate detection matrix at a 60% threshold

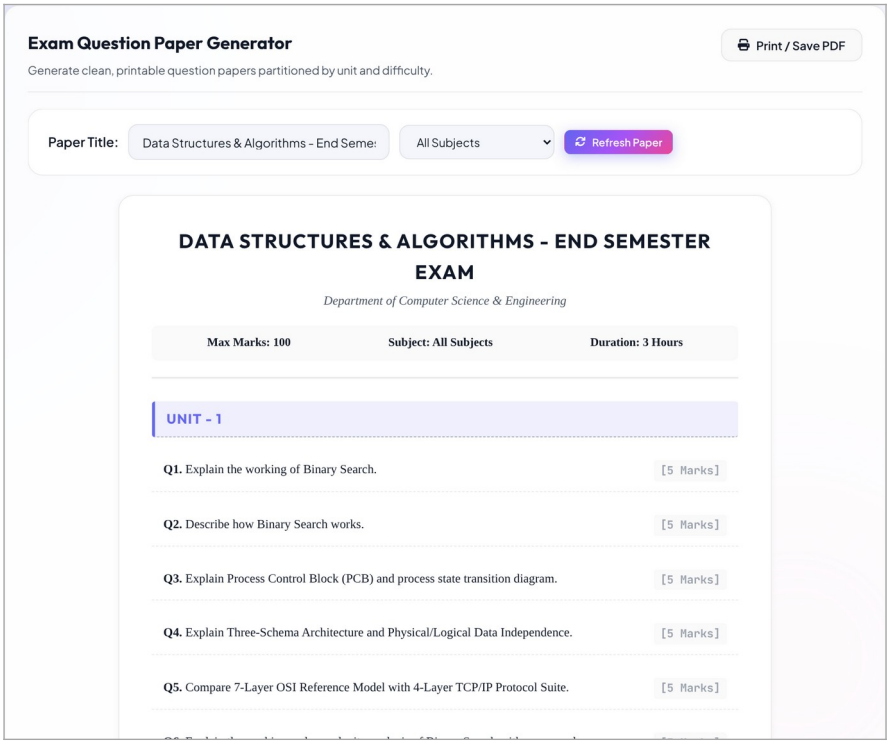


Fig. 5.20 Generated question paper grouped by unit

5.3.6 Role-based Access

Only administrators can add questions, through the dialog in Fig. 5.21. When a student logs in (Fig. 5.22), the Add, Import and Create Document buttons are hidden and each row of the table shows “View Only” instead of the edit and delete buttons.

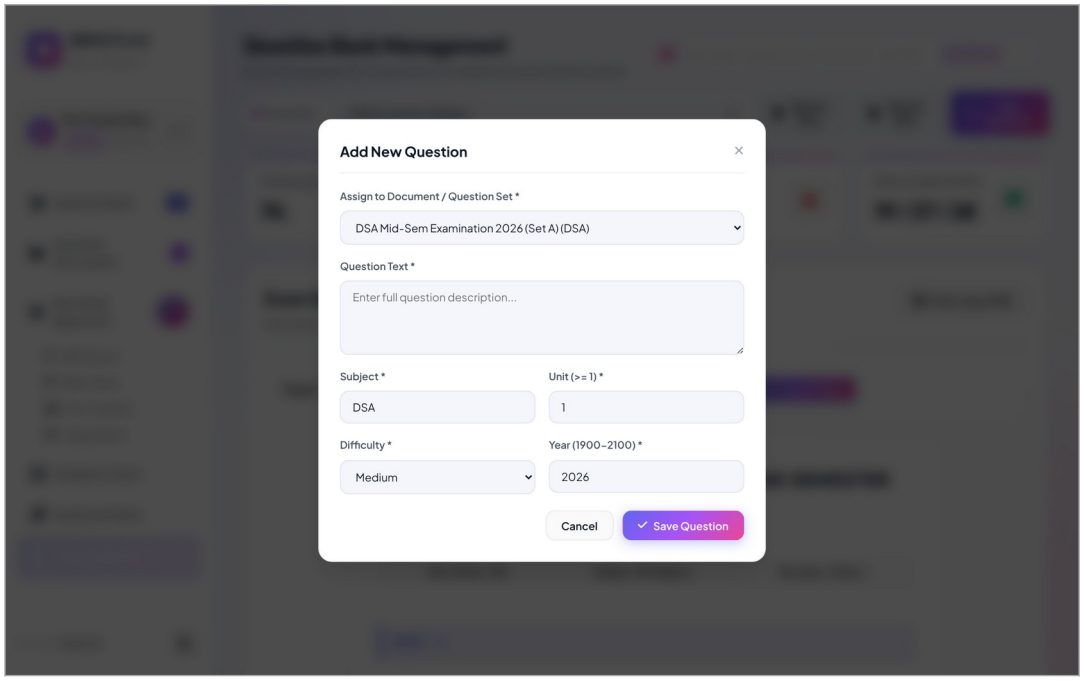


Fig. 5.21 Add Question dialog (administrator only)

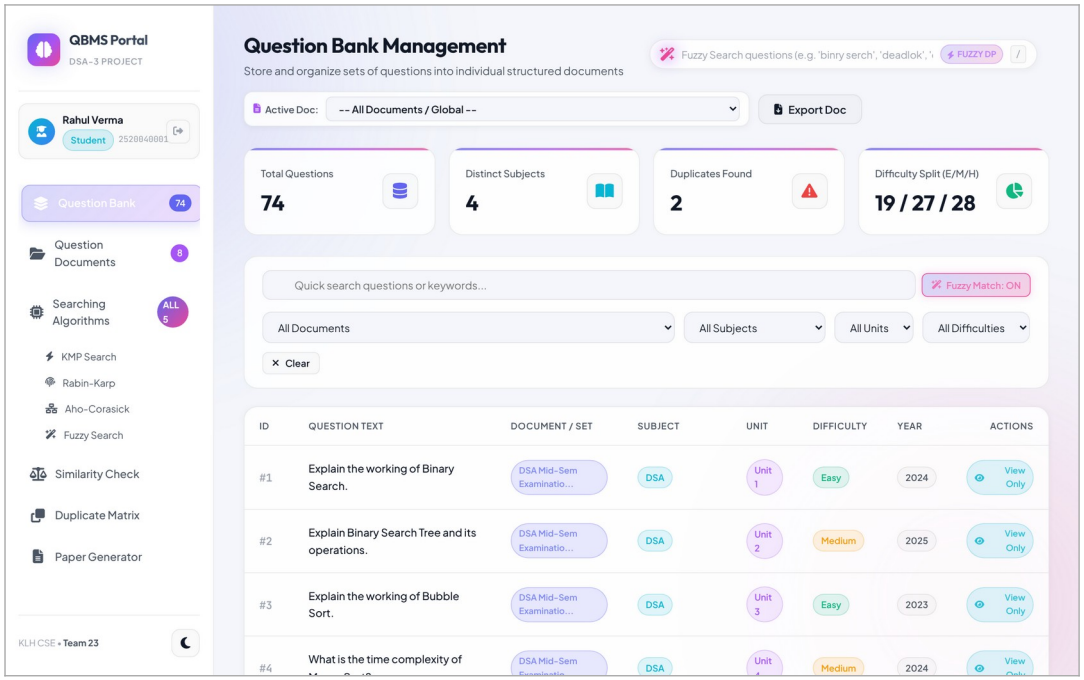


Fig. 5.22 Student view: read-only access without edit controls

5.4 Correctness Testing

To check the search algorithms, we compared them with a brute-force reference (`String.contains`, or a check at every position) on randomly generated inputs. The texts were 0–30 characters long and the patterns 1–5 characters, over the alphabet {a, b, c}. A small alphabet produces many repeated and overlapping matches, which are the cases most likely to expose bugs in LPS fallback, hash collisions or failure links.

Table 5.1 Randomised correctness tests against brute-force search

Implementation	Property checked	Cases	Mismatches
Java KMP, Java Rabin-Karp	Found / not found	200,000	0
Java Aho-Corasick (2 patterns)	Total number of occurrences	200,000	0
JS KMP, Rabin-Karp, Boyer-Moore-Horspool	List of all match positions	100,000 each	0
JS Aho-Corasick (2 patterns)	Total number of occurrences	100,000	0

All the exact-search implementations agreed with the reference in every case. The input validation was also tested by entering letters, blank lines and out-of-range values at each prompt; the console rejected them with a message and asked again.

5.5 Performance Comparison

Table 5.2 and Fig. 5.23 give the time to search all 102,000 questions for one keyword. Each value is from the third run, after the Java just-in-time compiler had warmed up. All three methods found the same number of matching questions.

Table 5.2 Single-keyword search over 102,000 questions (time in ms)

Keyword	Matching questions	KMP	Rabin-Karp	String.contains (JDK)
binary search	8,000	96.4	88.6	15.0
deadlock	4,000	57.1	75.4	21.7
tree	6,000	101.8	87.5	16.3

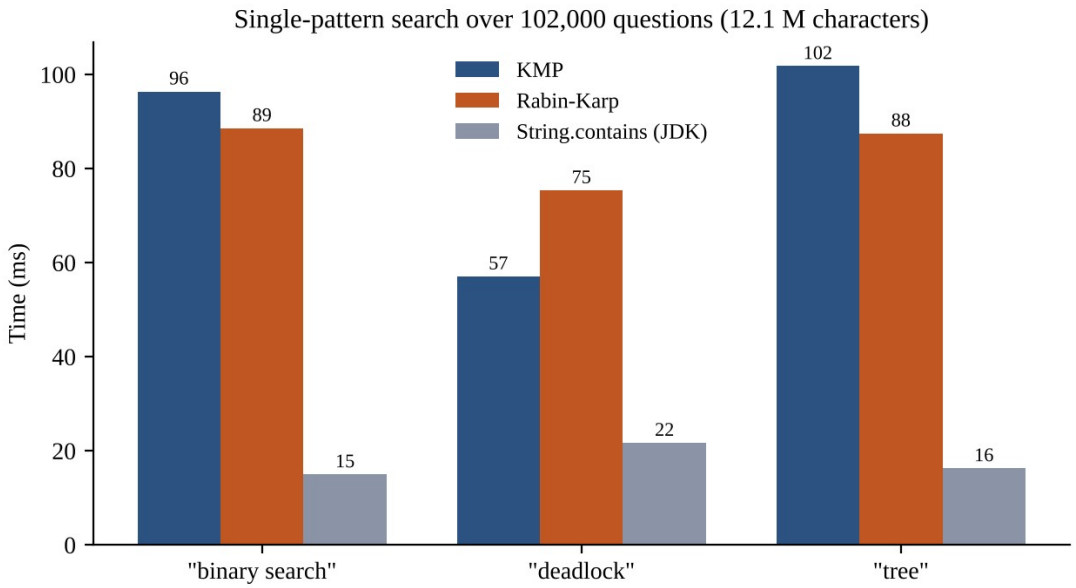


Fig. 5.23 Single-keyword search time over 102,000 questions

KMP and Rabin-Karp take similar times, both linear in the size of the corpus. The JDK's built-in

`String.contains` is about 4 to 6 times faster. This is expected: it is highly optimised native code, while our implementations call `toLowerCase()` and build a new LPS array or hash for every question. The value of KMP and Rabin-Karp in this project is their guaranteed linear behaviour and the internal data they expose for teaching, not raw speed on short questions.

Multi-keyword search shows a clear benefit (Fig. 5.24). Searching for three keywords with one Aho-Corasick automaton took **74.5 ms**, against **329.9 ms** for three separate KMP passes, a speed-up of about **4.4×**. Both methods found the same 14,000 matching questions. The advantage increases with the number of keywords, because Aho-Corasick still reads each question only once.

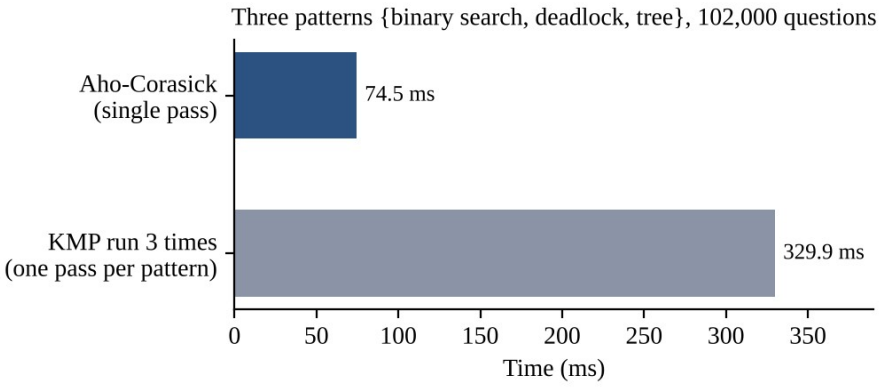


Fig. 5.24 Three-keyword search: Aho-Corasick vs. repeated KMP

Spurious hits in Rabin-Karp. Across the 51 questions and the three keywords, the modulus q = 101 produced **159 spurious hash matches** that had to be checked character by character. With q = 1,000,000,007 there were none. A small modulus is useful for showing hash values in the web visualiser, but a large prime is better for real use.

5.6 Duplicate Detection Results

Table 5.3 lists every pair flagged by the web client at a 60% similarity threshold on the 74-question bank (Fig. 5.19).

Table 5.3 Pairs flagged at a 60% similarity threshold (74 questions)

Similarity	Question pair (shortened)	Assessment
75.68%	Q1 “Explain the working of Binary Search” / Q3 “Explain the working of Bubble Sort”	False positive
75.21%	Q12 / Q46 Compare FIFO, Optimal and LRU page replacement (Q46 adds Belady’s Anomaly)	Genuine repeat
70.91%	Q11 / Q43 Banker’s Algorithm for deadlock avoidance (Q43 adds resource request)	Genuine repeat
68.47%	Q13 / Q47 Disk scheduling FCFS, SSTF, SCAN, C-SCAN (Q47 adds LOOK)	Genuine repeat
65.94%	Q21 / Q68 Distance vector routing, Bellman-Ford, count-to-infinity	Genuine repeat

Similarity	Question pair (shortened)	Assessment
62.50%	Q17 / Q56 ACID properties of transactions	Genuine repeat
61.34%	Q9 / Q36 Process Control Block and process state transitions	Genuine repeat

Six of the seven flagged pairs are real repeats across the built-in sets and the imported files, so the module finds the kind of duplicates that build up when question sets are merged. However, the results also show the weakness of character-level similarity. Questions 1 and 3 share a long template (“Explain the working of ...”) and are flagged although their topics differ. Questions 1 and 5 (“Explain the working of Binary Search” / “Describe how Binary Search works”) ask the same thing but score only **35.1%**, so they are missed (Fig. 5.6 and Fig. 5.18). Section 7.2 proposes word-based measures to address this.

6. CHALLENGES & LIMITATIONS

6.1 Issues Faced During Development

- **Case sensitivity** – a search for “binary” must also match “Binary” and “BINARY”.
- **Negative rolling hash** – in Rabin-Karp, subtracting the leading character's contribution can make the hash negative in Java and JavaScript, so equal windows would no longer produce equal hashes.
- **Overlapping and nested patterns** – in multi-pattern search, a plain trie walk does not report a short keyword that ends inside a longer one (“he” inside “she”).
- **Reporting every occurrence** – stopping at the first match was enough for the console, but the visualiser needed every position of the keyword.
- **Console input errors** – mixing `nextInt()` and `nextLine()` can leave newline characters in the buffer, and non-numeric input throws `NumberFormatException`.
- **Keeping two implementations consistent** – the same algorithms exist in Java and JavaScript, and they must give the same results.
- **Fuzzy matching of short queries against long questions** – edit distance between a two-word query and a forty-character question is always large.

6.2 Solutions Applied

- Text and patterns are converted to lower case in every algorithm before comparison.
- After each rolling-hash update, `q` is added whenever the hash is negative (`if (t < 0) t = t + q`).
- Failure links are built by breadth-first traversal, and each node copies the outputs of its failure node, so nested patterns are reported.
- The JavaScript KMP continues from `lps[j-1]` after each match, and the Java Aho-Corasick returns a list of `Match` objects with start and end indices.
- All console input is read with `nextLine()` and parsed inside `try/catch` blocks, with range checks and re-prompting.
- The JavaScript code uses the same constants as the Java code (d = 256, q = 101) and the same similarity formula, and both were tested against the same brute-force reference (Section 5.4).
- The web client's `bestSubstringDistance()` compares the query with windows of similar

length inside the question instead of with the whole question.

6.3 Limitations of the System

Testing showed the following limitations. We plan to address them before the final review.

- **The console does not load the JSON corpus** – the Java client works only on its 14 built-in questions; the four JSON files can be used only in the web client.
- **Whole-string fuzzy matching in Java** – as Fig. 5.5 shows, the console's fuzzy search does not tolerate typos in a short query. The sliding-window method of the web client has not yet been ported to Java.
- **Character-level similarity** – Levenshtein similarity misses reworded duplicates and flags questions that share a sentence template (Section 5.6).
- **Linear look-ups** – questions are kept in a list, so finding a question by ID or subject requires a scan. Hash-map indexes by ID, subject and difficulty, and sorting by year or difficulty, are not yet implemented.
- **Java search returns only true or false** – KMP and Rabin-Karp in Java report whether a question matches but not the match positions (the JavaScript versions do).
- **Client-side security only** – login checks only the format of the ID, and the administrator role is enforced in the browser. Any ID beginning with “EMP” gets administrator rights.
- **Unvalidated import** – imported JSON fields such as difficulty are inserted into the page without escaping, and a non-text difficulty value is saved and then causes an error each time the table is drawn. Imported data must be validated and escaped before it is stored.
- **Highlighting of overlapping keywords** – when several Aho-Corasick keywords overlap in one question, the highlighted text can show raw `<mark>` tags.
- **Rabin-Karp modulus** – q = 101 causes many spurious hits (Section 5.5).
- **Browser-only storage** – the bank is stored in one browser's `localStorage`, so it is not shared between users or devices.

7. CONCLUSION & FUTURE WORK

7.1 Conclusion

The Question Bank Management System shows how classical string-matching algorithms can solve a practical problem in academic administration. The project implements KMP, Rabin-Karp, Boyer-Moore-Horspool, Aho-Corasick and Levenshtein edit distance, and uses them for exact, multi-keyword and typo-tolerant search and for duplicate detection. The algorithms are available in a Java console client and in a browser-based web client with roles, documents, import and export, visualisations and paper generation.

Randomised testing found no mismatches between our implementations and brute-force search. On a 102,000-question corpus, Aho-Corasick searched three keywords about 4.4 times faster than repeated KMP. The duplicate detector found six genuine repeated questions among the 74 questions in the merged bank. The evaluation also identified clear limitations, the most important being character-level similarity and the Java client's limited fuzzy search and data loading. These give a specific plan for the next phase.

7.2 Future Work

- Load the JSON corpus in the Java client and report every match position from KMP and Rabin-Karp.
- Port the sliding-window fuzzy search to Java, and add a BK-tree over the vocabulary for fast typo suggestions.
- Add word-based similarity (Jaccard and cosine similarity over word sets or TF-IDF vectors, with stop-word removal and stemming) alongside Levenshtein distance, to catch reworded duplicates and ignore shared sentence templates.
- Index questions with hash maps keyed by ID, subject and difficulty, and keep a sorted structure (for example a `TreeMap`) by year, so that look-up and filtering do not need a full scan.
- Validate and escape all imported fields, and fix the highlighting of overlapping keywords.
- Use a large prime modulus (or double hashing) for Rabin-Karp outside the visualiser.
- Move storage and authentication to a server with a database (for example Spring Boot with MySQL), so that faculty can share one bank securely.
- Generate balanced question papers automatically, choosing questions to meet target marks

per unit and difficulty.

7.3 Skills Learned During Development

- Implementing and debugging string-matching algorithms (prefix functions, rolling hashes, tries with failure links, and DP tables) and analysing their complexity.
- Choosing data structures for a task: arrays and lists for storage, hash maps for trie transitions, queues for breadth-first construction, and 2-D arrays for dynamic programming.
- Testing algorithms systematically with randomised inputs against a brute-force reference, and measuring performance fairly with warm-up runs.
- Writing robust console programs in Java with input validation and clear menus.
- Building a single-page web application with JavaScript, the DOM, `localStorage` and the File API, and taking care of HTML escaping.
- Working as a team with Git and GitHub, and documenting a project in the university format.

REFERENCES

[1] D. E. Knuth, J. H. Morris and V. R. Pratt, “Fast pattern matching in strings,” *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.

[2] R. M. Karp and M. O. Rabin, “Efficient randomized pattern-matching algorithms,” *IBM Journal of Research and Development*, vol. 31, no. 2, pp. 249–260, 1987.

[3] A. V. Aho and M. J. Corasick, “Efficient string matching: An aid to bibliographic search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.

[4] R. S. Boyer and J. S. Moore, “A fast string searching algorithm,” *Communications of the ACM*, vol. 20, no. 10, pp. 762–772, 1977.

[5] R. N. Horspool, “Practical fast searching in strings,” *Software: Practice and Experience*, vol. 10, no. 6, pp. 501–506, 1980.

[6] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[7] R. A. Wagner and M. J. Fischer, “The string-to-string correction problem,” *Journal of the ACM*, vol. 21, no. 1, pp. 168–173, 1974.

[8] G. Navarro, “A guided tour to approximate string matching,” *ACM Computing Surveys*, vol. 33, no. 1, pp. 31–88, 2001.

[9] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009 (Chapter 32: String Matching).

[10] D. Gusfield, *Algorithms on Strings, Trees, and Sequences*. Cambridge, UK: Cambridge University Press, 1997.

[11] Oracle, “Java Platform, Standard Edition API Specification.” [Online]. Available: https://docs.oracle.com/en/java/javase/

[12] MDN Web Docs, “Web Storage API” and “FileReader.” [Online]. Available: https://developer.mozilla.org/